Chapter 1

Design

This chapter provides a comprehensive overview of the architectural and functional aspects of our RISC-V processor design. It details the structure of the processor's pipeline and explains how various functional units are seamlessly integrated into the core. The flow of instruction execution—from fetching to write-back—is described along with how the pipeline handles different scenarios. The processor is equipped with full interrupt and exception handling, which has been carefully designed and integrated into the pipeline control logic.

A key enhancement in our design is the implementation of a Memory Management Unit (MMU) that supports the Sv32 virtual memory scheme with a two-level page table structure and dynamic TLB (Translation Lookaside Buffer) updates. To enforce memory access control, we incorporated a Memory Protection Unit (PMP) that allows selective access to predefined physical memory regions, configurable during the boot process.

Additionally, a CLINT (Core Local Interruptor) controller has been integrated to manage software-generated and timer-based interrupts. For debugging, we developed a comprehensive Debug Unit that supports features such as single-stepping, hardware breakpoints, and the ability to inspect core registers and memory. This unit is interfaced with a custom-designed JTAG controller, enabling external debugging tools like GDB to connect and interact with the core during software development and testing.

1.1 Design Overview

Processor Core and ISA Support

- Implements a 32-bit RISC-V processor core compliant with the RV32G ISA, supporting integer (I), multiplication/division (M), atomic (A), single-precision (F), and double-precision (D) instructions.
- Designed with a 5-stage pipelined architecture comprising Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) stages.

Cache Architecture

- Equipped with separate Instruction and Data Caches, each with:
  - 8 KB capacity
  - 2-way set associative mapping
  - 32-byte block size
  - Least Recently Used (LRU) replacement policy
  - Write-back eviction strategy for efficient memory management
  - Software cache flush and Cache invalidation support

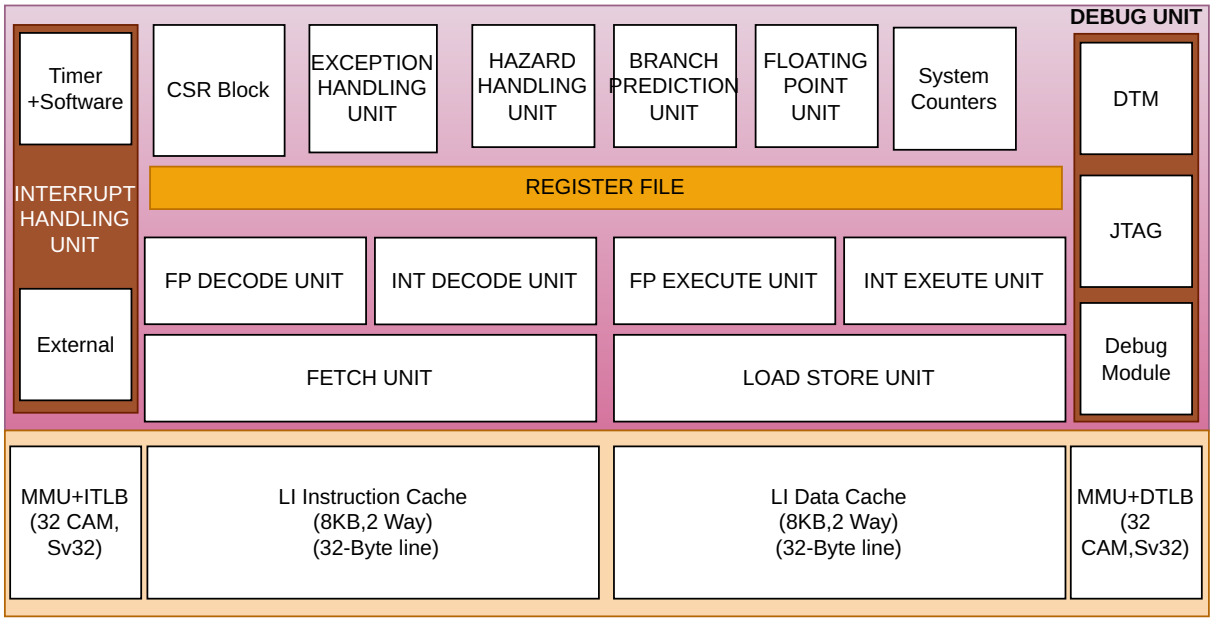


Figure 1.1: RV32G Processor architecture

Virtual Memory and Translation Lookaside Buffers (TLBs)

- Supports Sv32 virtual memory architecture with page table walking hardware.
- Instruction and Data TLBs (ITLB and DTLB) include:
  - 32 fully-associative CAM entries per TLB
  - Support for two-level page tables
  - Hardware-managed TLB filling for improved performance
  - Virtual address-based replacement policy

Memory Protection

- Incorporates a Memory Protection Unit (MPU) capable of managing up to 16 programmable memory protection regions.
- Enables region-based access control to enhance system security and task isolation.

Branch Prediction Unit

- Features a high-performance Branch Prediction Unit (BPU) including:
  - Branch Target Buffer (BTB) with 512 entries, 4-way set associative
  - Pattern History Table (PHT) with 2048 entries, utilizing 2-bit saturating counters
  - Return Address Stack (RAS) with 32 entries for accurate subroutine return address predictions

Interrupt and Exception Unit

- Provides support for software, timer and external interrupts.
- Machine level CSRs are implemented for internal exception handling
- Supports CLINT based interrupt architecture

Debug Capabilities

- Integrates JTAG controller for debugging using Serial Wire.
- Supprts single stepping, breakpoint and trace debug.
- Allows inspection of register state, CSRs and Caches using debugger

1.2 Core Architecture

The processor is a 32-bit implementation of the RISC-V architecture, fully compliant with the RV32G standard [?]. The "G" extension signifies that the processor supports a comprehensive instruction set including the base integer instructions (I), multiplication and division (M), atomic operations (A), single-precision floating-point (F), and double-precision floating-point (D). This makes the core suitable for both general-purpose computing and applications requiring numerical processing or concurrency control. The processor is designed for high efficiency and real-time capability, making it suitable for embedded systems and operating system-level workloads. It also supports RTOS execution and includes a debug module capable of halting execution, single-stepping through instructions, and executing commands from a program buffer or via abstract command mechanisms. The processor microarchitecture is given in figure 1.2.

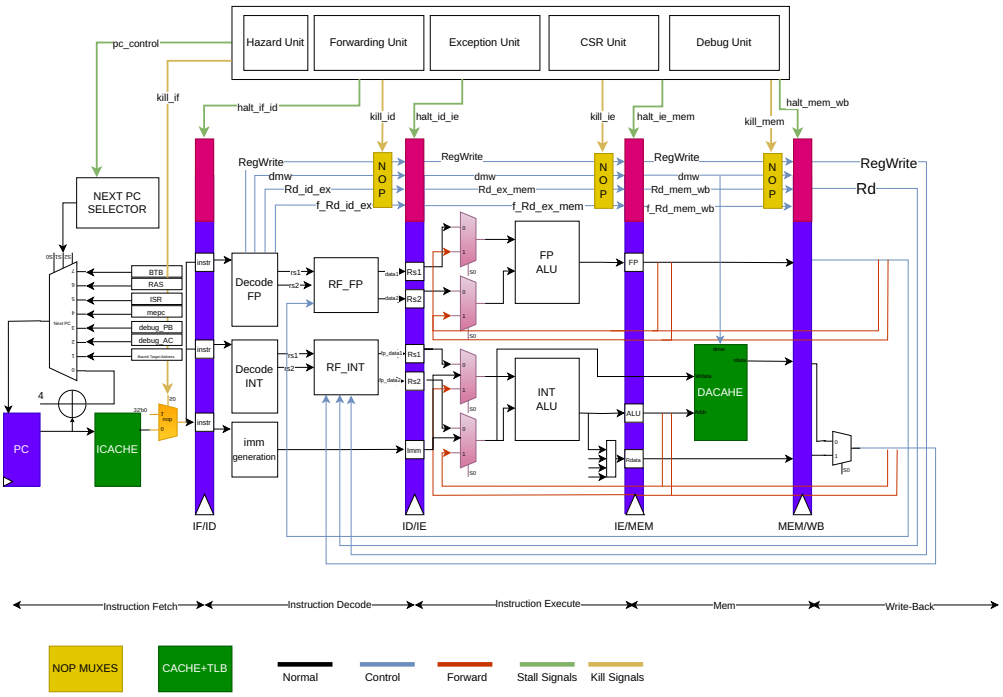


Figure 1.2: RV32G Processor micro-architecture

Key Architectural Components:

1. Pipeline Architecture: Implements a 5-stage pipeline comprising Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB), allowing instruction-level parallelism and efficient execution flow.

2. Branch Prediction Unit: Improves processor performance by guessing the outcome of branch instructions to reduce pipeline stalls.

3. Memory Subsystem: Features instruction and data caches (each 8KB, 2-way set associative), with write-back eviction and LRU replacement. Includes ITLB/DTLB

with Sv32 virtual memory support and a memory protection unit (PMA/PMP) supporting up to 64 regions.

4. Control and Status Registers (CSR): Fully compliant with the RISC-V privileged specification, enabling software to interact with exception handling, interrupt control, and processor state.

5. This processor supports external interrupts to be service as per the RISC-V Privileged specification and hardware interrupt handling.

6. Exception Handling Unit: Detects and manages synchronous events (e.g., illegal instructions, page faults), redirecting control flow via trap handlers configured through the CSR block.

7. Debug Support: Integrated debug module enables full system observability, including features such as single-step execution, halt/resume control, and execution of commands through the program buffer or abstract command interface.

1.3 Pipeline

The RV32G processor adopts a classic 5-stage pipeline architecture as shown in the below figure 1.3 —Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB)—augmented with additional stall, kill, and forwarding logic to manage hazards and ensure correct instruction execution. Each stage is designed for optimal throughput while maintaining support for advanced features like floating-point operations, and memory-mapped I/O.

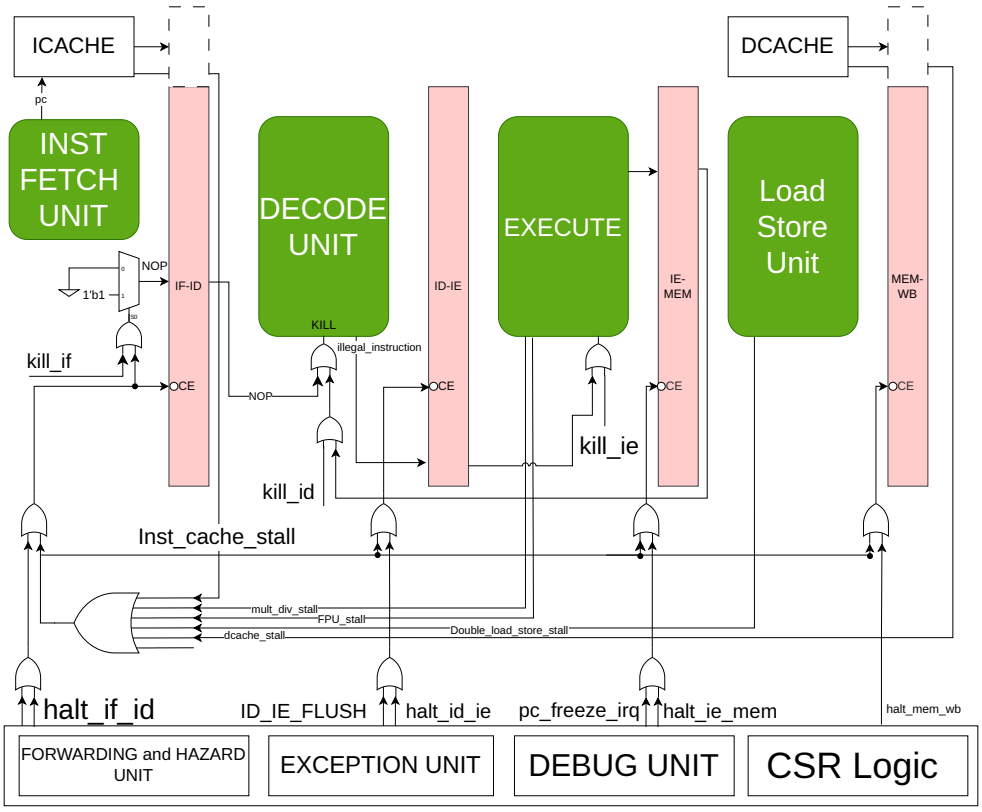


Figure 1.3: Pipeline Architecture

1. **Instruction Fetch (IF)**: The Instruction Fetch stage is responsible for retrieving instructions from memory using a tightly coupled *ICACHE* unit. Instructions are aligned on 32-bit word boundary and **fetch one instruction per cycle**, assuming the instruction-side memory system (e.g., instruction cache) is ready. The *inst_ cache_ stall* signal may be asserted in this stage when the instruction cache cannot immediately provide the requested data, resulting in a stall across the pipeline. Control signals like *kill_ if* and *halt_ if_ id* can be asserted here to invalidate or stall instructions, typically due to branch mispredictions, exceptions, or interactions with the debug unit.

2. **Instruction Decode (ID)**: The Instruction Decode stage interprets the opcode and operands from the fetched instruction and initiates register file reads for

source operands. This stage also interacts with the forwarding logic to resolve data hazards by reusing results from later stages when needed. If dependencies cannot be bypassed, stalls such as *halt_id_ie* may be introduced. Control and exception-related operations (e.g., CSR accesses) are decoded here, and *kill_id* can invalidate instructions due to misprediction or pipeline flushes.

3. **Execute (EX)**: The Execute stage performs the actual computation required by the instruction. It houses key functional units including:

   - The Arithmetic Logic Unit (ALU)
   - The Multiplier-Divider Unit (MDU) for integer multiply/divide instructions
   - Branch conditions are evaluated in this stage, and branch decisions are made here. Multi-cycle operations—such as multiplication, division, and certain floating-point operations—will stall the EX stage using control signals like *mult_div_stall* or *FPU_stall* until the operation completes. Data dependencies may also trigger pipeline stalls if forwarding is insufficient, particularly when dealing with memory or floating-point results.

4. **Memory Access (MEM)**: In the Memory Access stage, load and store instructions interact with the data cache or memory-mapped I/O peripherals. Address values are passed from the EX stage, and the memory hierarchy returns or receives data accordingly. This stage handles stalls due to:

   (a) Double load/store hazards (e.g., *double_load_store_stall*)

   (b) Data cache unavailability (*dcache_stall*)

   Integration with the PMA/PMP unit ensures that memory accesses respect protection and region-based access policies. The exception unit is actively involved here in validating memory accesses and raising faults if necessary.

5. **Write-Back (WB)**: The Write-Back stage completes the execution cycle by writing the results of ALU operations, memory loads, multiplication/division outputs, and floating-point computations back into the register file. This stage marks the end of the instruction's lifetime in the pipeline and also supports interaction with the debug module if a single-step or halt condition was in effect. The *halt_mem_wb signal* may pause this stage if there is an unresolved pipeline dependency, debug request, or flush event from exception or trap logic.

6. **Control and Coordination Units**

   - **Forwarding and Hazard Unit**: Ensures data is forwarded between pipeline stages when possible and detects data hazards that require stalls or data forwarding.
   - **Exception Unit**: Detects illegal instructions and synchronous faults, and generates appropriate trap/interrupt control.
   - **CSR Logic**: Manages privileged state and communication with system-level software, integrating closely with the exception unit.

- **Debug Unit**: Introduces halts, breakpoints, or kill signals to support single-stepping, abstract command execution, and visibility into processor state.

7. **Kill and Halt Control Signals**: To resolve pipeline hazards or invalid instruction flows, the following kill and halt signals are used:

   (a) **Kill Signals**: *kill_if*, *kill_id*, *kill_ie* Used to invalidate instructions in various stages (e.g., after a branch misprediction or exception).

   (b) **Halt Signals:**: *halt_if_id*, *halt_id_ie*, *halt_ie_mem*, *halt_mem_wb* Temporarily stop progression between stages to prevent incorrect execution or allow time for dependent data.

   (c) *PC_CONTROL_IRQ*, *PC_FREEZE_IRQ*, *ID_IE_FLUSH* signals are asserted by the exception unit to halt the IF stage for fetching instructions from the ISR address and let other stages to complete the current instruction currently in the pipeline. This ensures less interrupt overhead.

The Table 1.1 shows the cycle counter per instructions which are executed in the pipeline.

Instruction Type	**Cycles**	**Description**
Integer Computational	1	Integer Computational Instructions are defined in the RISCV-V RV32I Base Integer Instruction Set.
CSR Access	1	CSR Access Instruction are defined in 'Zicsr' of the RISC-V specification.
Load/Store	1	Load/Store is handled in 1 bus transaction using both EX and WB stages for 1 cycle each. Misaligned transfers are not supported and will raise and exception
Multiplication	1 (mul) 4 (mulh, mulhsu, mulhu)	RV32G uses a single-cycle 32-bit x 32-bit multiplier with a 32-bit result. The multiplications with upper-word result take 4 cycles to compute.
Division Remainder	3 - 35 3 - 35	The number of cycles depends on the divider operand value (operand b), i.e. in the number of leading bits at 0. The minimum number of cycles is 3 when the divider has zero leading bits at 0 (e.g., 0x8000000). The maximum number of cycles is 35 when the divider is 0
Jump/ Branches	1 (Branch predicted correctly) 3 (Misprediction)	Jumps and Branches are taken in EX stage and hence 2 stages needs to be flushed
mret/ ECALL/ EBREAK	2	Mret is performed in the ID stage. Upon an mret the IF stage is flushed. The new PC request will appear on the instruction-side memory interface the same cycle the mret instruction is in the ID stage.

Table 1.1: Cycle count per instruction

1.4 System Control Unit (SysCtl)

The SysCtl Unit acts as a central controller for events that affect all other units, like interruptions and exceptions. The Control and Status Registers in RISC-V privileged ISA control the essential core settings. Hence, these CSRs are also managed by SysCtl Unit. Figure 1.4 shows the logical block diagram for System Control (SysCtl) Unit.

The RISC-V architecture, as detailed in the "RISC-V Instruction Set Manual Volume II: Privileged Architecture Document Version 20211203," provides a comprehensive framework for handling exceptions and interrupts. For our RV32G core, the exception unit is a critical component that manages these events according to the specifications outlined in the privileged document

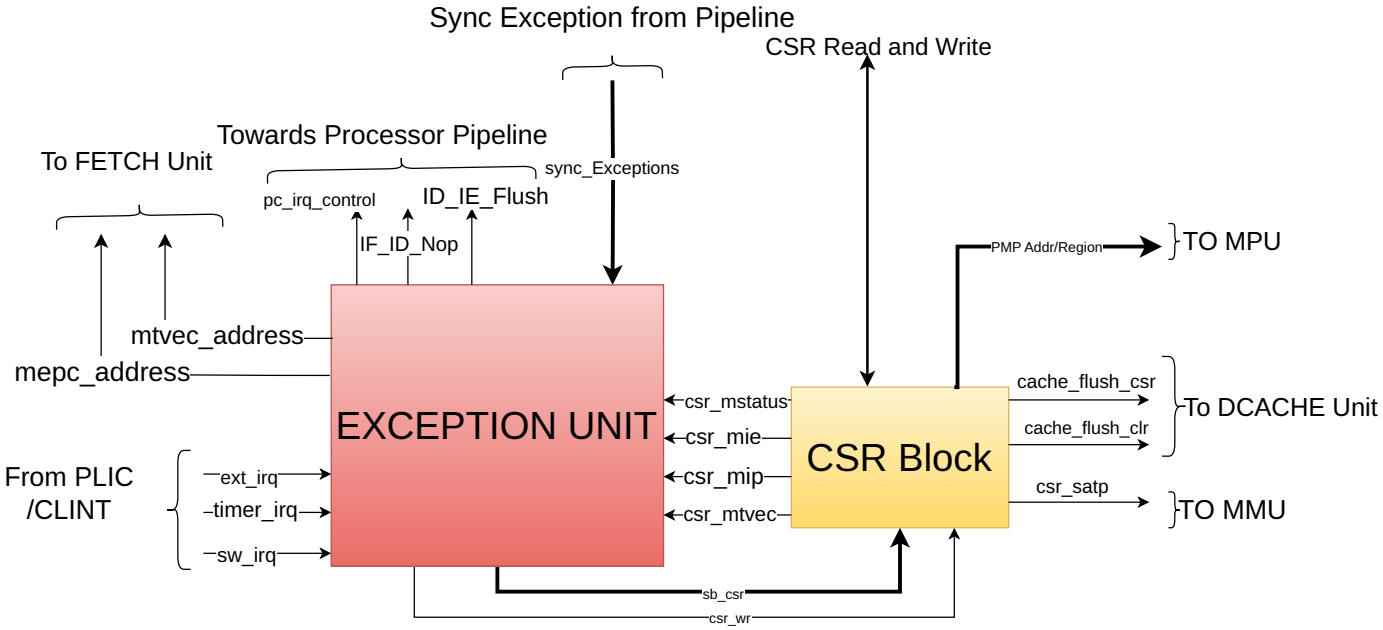


Figure 1.4: System Control Unit

1.5 EXCEPTION UNIT

The exception unit in RV32G core is responsible for managing and handling exceptions and interrupts according to the RISCV Privilaged specifications [?]. It interfaces with various components of the processor, including the pipeline, CSR (Control and Status Register) block, and memory units. The primary functions of the exception unit include:

- Detecting and handling synchronous and asynchronous exceptions.
- Updating relevant CSRs to reflect the state of the processor during an exception or interrupt.

- Ensuring proper handling and resolution of exceptions to maintain system stability and security.

It handles the exception and interrupts for the Core. In the RISC-V architecture, exceptions and interrupts are both considered traps, but they differ fundamentally in their source and behavior. Exceptions are synchronous events that arise due to the execution of specific instructions—such as ***illegal operations, memory access faults, or system calls***—and they occur predictably during the instruction flow. These are typically used to signal errors or specific conditions that require immediate attention by the processor. **Interrupts**, on the other hand, are asynchronous events triggered by external or internal devices such as **timers, I/O peripherals, or software signals**. They occur independently of the current instruction execution and can happen at any point between instructions. While exceptions help the processor respond to internal execution issues, interrupts allow it to handle real-time events and interact with the outside world efficiently. Each of these exceptions have a specific code in the ***mcause*** register in the CSR unit. When any of these occurs, ***mcause*** register is loaded with the value assigned to the corresponding exception, current PC is stored in ***MEPC*** register.

The ***ECALL*** instruction is used to trigger a system call or exception, typically for transitioning control to the operating system or firmware, such as entering a handler in machine mode. Since it causes an exception, all instructions before the ECALL must complete before it is processed. The instruction is recognized during the Decode stage, but it will only proceed if no prior branch is taken—if a branch is taken, the Decode stage is flushed, and the ECALL will not execute. If a branch is not taken, the PC is loaded with the *mtvec* value and the decode and execute unit's are flushed and NOP is inserted.

MRET Instruction is used to return from an interrupt routine. PC is loaded with the value of mepc and normal program execution resumes.

The CSR (Control and Status Register) block in a RISC-V processor plays a pivotal role in exception and interrupt handling by maintaining critical system-level information. One of the key registers is *csr_mstatus*, which monitors the processor's current state, including the privilege level, interrupt enable settings, and context stack, ensuring a smooth transition during trap entry and return. The *csr_mepc* register records the program counter (PC) of the instruction that was executing at the time the exception occurred, allowing the processor to resume execution from the correct point after handling the trap. The *csr_mie* register manages the interrupt enable bits, determining which specific interrupts are allowed to trigger a trap, while the *csr_mip* register indicates pending interrupts that await service. Finally, the *csr_mtvec* register holds the base address for the trap vector, guiding the processor to the appropriate handler routine during exceptions or interrupts.

Exception Handling

When entering an interrupt/exception handler, the core sets the *mepc* CSR to the current program counter and saves *mstatus.MIE* to *mstatus.MPIE*. All exceptions cause the core to jump to the base address of the vector table in the *mtvec* CSR. Interrupts are handled in either non-vectored mode or vectored mode depending on the value of *mtvec.MODE*. In non-vectored mode the core jumps to the base address of the vector table in the mtvec

CSR. In vectored mode the core jumps to the base address plus four times the interrupt ID. Upon executing an mret instruction, the core jumps to the program counter previously saved in the *mepc* CSR and restores *mstatus.MPIE* to *mstatus.MIE*.

1.5.1 Exception Detection

- Synchronous Exceptions: These are detected during the execution of instructions, such as illegal instructions, address misaligned, or access faults.
- Asynchronous Exceptions: These include external interrupts, timer interrupts, and software interrupts. These are detected through external signals like *ext_irq*, *timer_irq*, and *sw_irq*.

Below Table 1.2 shows the current implemented interrupts and exception in the core.

Exceptions	**I/E**	**Name**	**Description**
Asynchronous	I	Machine Software Interrupt	Occurs when core writes 1 in MSIP register
Asynchronous	I	Machine Timer Interrupt	Occurs when core writes 1 in MTIMECMP register
Asynchronous	I	Machine External Interrupt	Occurs when external sources like peripheral, I/Os request for the interrupt

Table 1.2: Interrupts for RV32G

Exceptions	I/E	Name	Description
Synchronous	E	inst_addr_misaligned	When PC is not aligned to 32 bit word boundary
Synchronous	E	inst_access_fault	When PC tries to read a forbidden region in Instruction Memory
Synchronous	E	illegal_instruction	If the current instruction in the decode stage is illegal
Synchronous	E	csr_ro_wr	If we attempt to write a read-only CSR
Synchronous	E	sys[‘IS_EBREAK]	If we encounter EBREAK[1]
Synchronous	E	load_misaligned	If the Load address is not aligned to 32-bit word boundary
Synchronous	E	load_access_fault	If we try to load from not-allowed memory region
Synchronous	E	store_misaligned	If the Store address is not aligned to 32-bit word boundary
Synchronous	E	store_access_fault	If we try to store to a not-allowed memory region
Synchronous	E	sys[‘IS_ECALL]	If we encounter ECALL instruction
Synchronous	E	instruction_page_fault	If we do not have the correct R/X permission for that page
Synchronous	E	load_page_fault	If we do not have the correct R/W/X permission for that page
Synchronous	E	inst_dec_error	If we encounter a wrong instruction(Unspecified opcode)

Table 1.3: Exceptions for RV32G

1.5.2 CSR Updates

The below algorithm 1.1 shows how the CSR registers are updated when the core enters in the trap mode.

Algorithm 1.1 Exception Entry Sequence

/* Synchronous Exception Entering Sequence */
1. mepc <- Current PC
2. mcasue <- Specific cause code for that exception
3. mtval <- Value related to that exception
4. mstatus.mie <- 0 // Disable Global Interrupt Enable
5. mstatus.mip <- 1 // Pending Interrupt flag is raised
PC <- mtvec_address

- **mepc**: When a trap is taken into M-mode, *mepc* is written with the virtual address of the instruction that was interrupted or that encountered the exception.
- **mcause**: When a trap is taken into M-mode, *mcause* is written with a code indicating the event that caused the trap.
- **mtval**: When a trap is taken into M-mode, *mtval* is either set to zero or written with exception-specific information to assist software in handling the trap. Eg.
  - When a misaligned load or store operation results in an access-fault or page-fault exception, the *mtval* register is updated with a nonzero value that holds the virtual address corresponding to the part of the access that triggered the fault.
  - When an instruction access-fault or page-fault exception occurs, then *mtval* will contain the virtual address of the of the instruction that caused the fault, while mepc will point to the beginning of the instruction.
- **mstatus**: The *mstatus* register is updated to reflect the new state. For example, the MPIE bit is set to the current MIE state, and MIE is cleared.
- **mtvec**: The *mtvec* register is used to determine the base address for the trap vector. It has a direct mode and a vectored mode. In direct mode it always points to the base address of the vector table. In vectored mode, the mtvec registers points to the base_address + offset, determined by the mcause code of the interrupt that occured.

1.5.3 Trap Vector Calculation

The *mtvec* register is shown in the figure 1.5 below. The trap vector is calculated based on the *mtvec register* and the *exception cause*. The processor jumps to the appropriate handler address to handle the exception. The figure 1.5 shows the format of mtvec register.

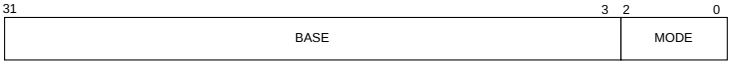


Figure 1.5: Mtvec register

When MODE=Direct, all traps into machine mode cause the PC to be set to the address in the BASE field. When MODE=Vectored, all synchronous exceptions into machine mode cause the PC to be set to the address in the BASE field, whereas interrupts cause the PC to be set to the address in the BASE field plus four times the interrupt cause number.

1.5.4 Mcause codes

The *mcause* register is shown in the figure 1.6 below. In the RISC-V privileged specification [?], the *mcause* CSR (Machine Cause Register) holds the reason for the last trap (which can be an interrupt or an exception). This register helps the system determine how to handle the event appropriately.

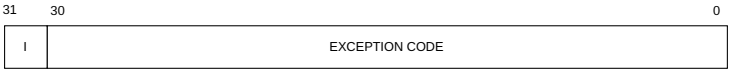


Figure 1.6: Mcause register

The most significant bit of mcause (bit XLEN-1) indicates whether the trap was caused by an interrupt (1) or by a synchronous exception (0). The remaining bits encode the specific cause code, which identifies the exact nature of the trap.

The table 1.5 shows the *mcause* codes corresponding to each exception or interrupt. These codes are essential in designing the trap handler logic, where different actions can be taken based on the type and source of the trap.

Interrupt	**Exception Code**	**Description**
1	3	Machine Software Interrupt
1	7	Machine Timer Interrupt
1	11	Machine External Interrupt
0	0	Instruction Address Misaligned
0	1	Instruction Access fault
0	2	Illegal Instruction
0	3	Breakpoint
0	4	Load Address misaligned
0	5	Load Access fault
0	6	Store/AMO Address misaligned
0	7	Store/AMO Access fault
0	11	ECALL from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	24	Instruction Deocde Error

Table 1.5: Mcause exception codes

1.5.5 Pipeline Control

The exception unit may signal the pipeline to flush and ensure that no further instructions are executed until the exception is handled. This is crucial to prevent further exceptions or incorrect state changes.

When a synchronous (e.g., ***msie***, ***mtie***, ***meie***) or asynchronous exception occurs, the ***trap_ en*** signal is asserted, stalling the pipeline to halt instruction flow. Relevant CSR registers are updated with exception details, and the *mtvec* register provides the vector table base address, which is loaded into the PC to redirect execution to the exception handler. Upon encountering the MRET instruction in the decode stage, the PC is restored from the saved address, and the core resumes normal execution.

Handling interrupts in hardware offers several key advantages over software-based handling, particularly in real-time and embedded systems where timing is critical. Hardware interrupt handling significantly reduces interrupt latency, which is the time between the occurrence of an interrupt and the start of the corresponding interrupt service routine (ISR). This is crucial in real-time systems, where deterministic and timely responses are required.

With hardware support, interrupts can be detected and serviced almost immediately, without the overhead of polling or extensive context checks in software. The figure 1.7 shows the integration of exception unit in with the pipeline

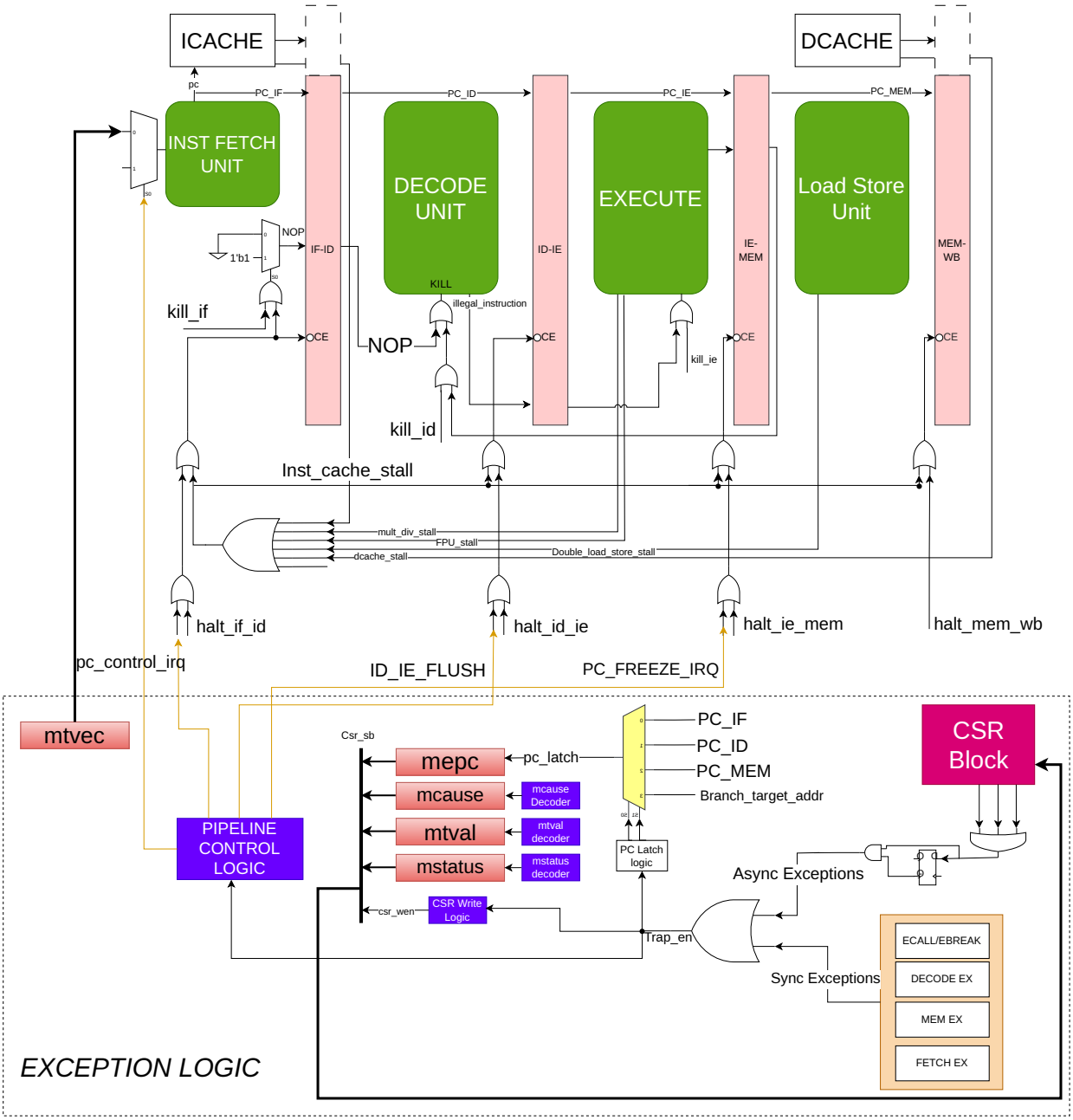


Figure 1.7: Exception Unit pipeline integration

1.5.6 CONTROL AND STATUS REGISTERS

1.5.7 Machine Mode CSR

The Control and Status Register (CSR) architecture in RISC-V, as defined in the RISC-V Privileged Specification[?], incorporates a structured address decoding mechanism to manage access permissions and privilege levels. Specifically, the upper four bits of the

12-bit CSR address field (csr[11:8]) are reserved for this purpose. The top two bits (csr[11:10]) determine the access type: values 00, 01, or 10 indicate that the register is read/write, while 11 designates a read-only register. The subsequent two bits (csr[9:8]) specify the minimum privilege level required to access the CSR—00 for User mode, 01 for Supervisor mode, 10 for Hypervisor mode (if supported), and 11 for Machine mode. This encoding ensures that each CSR is accessed only by the appropriate privilege level and with the correct permissions, thereby enhancing system security and maintaining controlled interaction between hardware and software. Table 1.6 shows the machine mode CSR implemented in our core.

Name	Address	Description
mstatus	0x300	Machine Status (lower 32 bits).
mie	0x304	Machine interrupt enable
mtvec	0x305	Machine Trap Vector address
mepc	0x341	Machine Exception Program Counter
mcause	0x342	Machine Trap Cause
mtval	0x343	Machine Trap Value
mip	0x344	Machine interrupt pending

Table 1.6: Machine mode CSR map

The following CSRs are essential for managing interrupts and exceptions in the RISC-V architecture: *mstatus* (0x300) holds the machine status, including global interrupt enable and privilege mode information; *mie* (0x304) allows enabling of specific interrupts; *mtvec* (0x305) contains the base address for the trap handler; *mepc* (0x341) stores the program counter at the time of an exception; *mcause* (0x342) indicates the cause of the trap; *mtval* (0x343) provides fault-specific data like the address or instruction that caused the exception; and *mip* (0x344) reflects which interrupts are currently pending. These CSRs are initialized by software immediately after the processor comes out of reset to configure the core's behavior in response to traps and interrupts.

1.5.8 Debug CSR

We have implemented the core debug registers (*dcsr*, *dpc*, *dscratch0*, and *dscratch1*) in accordance with the official RISC-V Debug Specification[**?**]. These registers are accessible only when the core enters Debug Mode and can be used through standard CSR instructions or abstract debug commands. Any attempt to access these registers outside Debug Mode, or on a core where they are not implemented, results in an illegal instruction exception. This ensures compliance with the RISC-V debug architecture and provides essential support for system-level debugging. Table 1.7 shows the address of debug CSRs.

Name	Address	Description
dcsr	0x7b0	Machine Status (lower 32 bits).
dpc	0x7b1	Machine interrupt enable
dscratch0	0x7b2	Machine Trap Vector address
dscratch1	0x7b3	Machine Exception Program Counter

Table 1.7: Debug CSRs

1.5.8.1 Debug Control and Status (dcsr, at 0x7b0)

Upon entry into Debug Mode, *v* and *prv* are updated with the privilege level the hart was previously in, and *cause* is updated with the reason for Debug Mode entry. Other than these fields and *nmip*, the other fields of *dcsr* are only writable by the external debugger. Since we only support the machine mode, *v* and *prv* will behardwired to 0. The figure 1.8 shows the format of DCSR register

Figure 1.8: DCSR CSR

During the debug mode entry, the cause of entry is written in the dcsr[cause] bit. The specification describes the priority between different debug entry sources.

1.5.8.2 Debug PC (dpc, at 0x7b1):

Upon entry to debug mode, *dpc* is updated with the virtual address of the next instruction to be executed. The writability of *dpc* follows the same rules as *mepc* as defined in the Privileged Spec. When we resume, the heart's PC is restored with the value in *dpc*. The figure 1.8 shows which address needs to be stored in DPC, while entering the debug mode.

Cause	dpc value
ebreak	Address of the ebreak instruction
single step	Address of the instruction that would be executed next if no debugging was going on. Ie. pc + 4 for 32-bit instructions that don't change program flow, the destination PC on taken jumps/branches, etc.
halt request	Address of the next instruction to be executed at the time that debug mode was entered

Table 1.8: Virtual address in DPC upon Debug Mode Entry

1.5.8.3 Debug Scratch Register 0/1 (dscratch0/1, at 0x7b2/0x7b3):

The *dscratch0* and *dscratch1* registers serve as temporary general-purpose registers that can be used by debugging tools or software during debug operations. Since the debugger may need to perform various operations without interfering with the normal state of the core, these scratch registers offer safe storage for intermediate values, calculations, or context data. They are especially useful for saving and restoring register values when single-stepping or inspecting memory and allow for flexible debug routines without corrupting the core's execution state.

1.5.9 PMP Configuration CSRs

In our design, the physical memory can be divided into a maximum of 16 configurable regions, allowing fine-grained control over memory access. Each region is defined using a PMPADDRx register, which stores the base or top address depending on the selected addressing mode. To specify access permissions and region attributes, each memory region is associated with an 8-bit configuration field that includes read, write, and execute permissions, the addressing mode, and a lock bit to prevent further modification. Since each *pmpcfg* register holds 8 bits per region, a single register can manage configuration for up to 4 regions. Therefore, a total of 4 PMPCFG registers are implemented in our core to support all 16 regions. This setup enables flexible and secure memory partitioning aligned with the RISC-V privilege specification. The table 1.9 shows address map of PMPCFGx and PMPADDRy CSRs in our design.

Name	Address	Description
PMPCFG0	0x3A0	PMP configuration 1
PMPCFG1	0x3A1	PMP configuration 2
PMPCFG2	0x3A2	PMP configuration 3
PMPCFG3	0x3A3	PMP configuration 4
PMPADDR0	0x3B0	PMP address region 1
PMPADDR1	0x3B1	PMP address region 2
PMPADDR2	0x3B2	PMP address region 3
PMPADDR3	0x3B3	PMP address region 4
PMPADDR4	0x3B4	PMP address region 5
PMPADDR5	0x3B5	PMP address region 6
PMPADDR6	0x3B6	PMP address region 7
PMPADDR7	0x3B7	PMP address region 8
PMPADDR8	0x3B8	PMP address region 9
PMPADDR9	0x3B9	PMP address region 10
PMPADDR10	0x3BA	PMP address region 11
PMPADDR11	0x3BB	PMP address region 12
PMPADDR12	0x3BC	PMP address region 13
PMPADDR13	0x3BD	PMP address region 14
PMPADDR14	0x3BE	PMP address region 15
PMPADDR15	0x3BF	PMP address region 16

Table 1.9: PMP Configuration CSR

1.5.10 Custom CSR

We support cache flush feature in dcache. To flush the cache, write 1'b1 on this register. The table 1.10 shows the address of Cache_flush CSR in our design.

Name	Address	Description
cahce_flush	0x400	Cache flush

Table 1.10: Custom CSRs

1.6 Interrupt Subsystem

This core utilizes a CLINT-based interrupt architecture, where the irq_i[31:0] interrupt signals are managed through the *mstatus*, *mie*, and *mip* CSRs. The core designates the upper 16 bits (irq_i[31:16]) of the *mie* and *mip* registers for custom interrupt handling, reflecting an intentional extension to the standard RISC-V CLINT model. The figure 1.9 shows the interrupt platform integrated with the core.

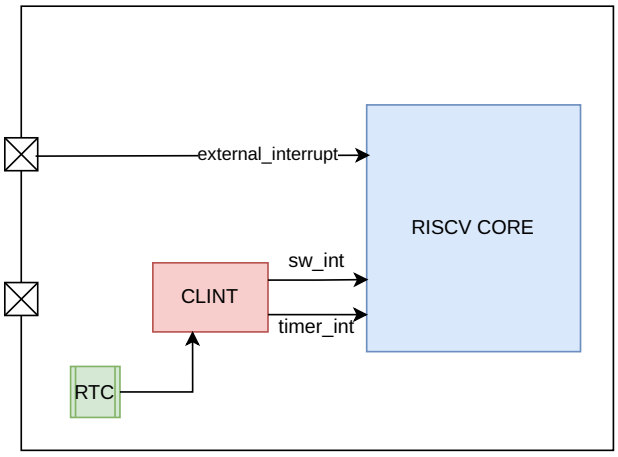


Figure 1.9: Interrupt Platform

For any irq_i[31:0] interrupt to become active, both the global interrupt enable (MIE) bit in mstatus and the corresponding individual enable bit in the mie register must be set.

The interrupts needs to be enabled first during the boot configuration. If the corresponding bit in MIE is 1, then if that interrupt occurs, it will make the MIP bit as 1 indicating that interrupt is pending and needs to be serviced. External interrupt bit is cleared once it is serviced, but software and timer bits needs to be cleared by writing zero in MSIP or MTIMECMP register. Figure 1.10 shows the machine interrupt enable (MIE) and machine interrupt pending (MIP) register.

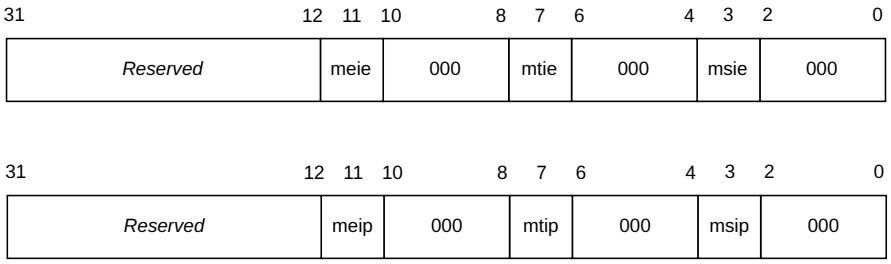


Figure 1.10: mie and mip CSR register

Table 1.11 describes the interrupt interface used for the CLINT mode interrupt architecture.

Signals	Direction	Description
irq_i[31:16]	Input	Active high, level sensistive interrupt inputs. Custom extension.
irq_i[15:12]	Input	Reserved. Tie to 0.
irq_i[11]	Input	Active high, level sensistive interrupt input. Referred to as Machine External Interrupt (MEI)
irq_i[10:8]	Input	Reserved. Tie to 0.
irq_i[7]	Input	Active high, level sensistive interrupt input. Referred to as Machine Timer Interrupt (MTI)
irq_i[6:4]	Input	Reserved. Tie to 0.
irq_i[3]	Input	Active high, level sensistive interrupt input. Referred to as Machine Software Interrupt (MSI)
irq_i[2:0]	Input	Reserved. Tie to 0.

Table 1.11: CLINT mode architecture interface signals

When one of the assigned interrupts occurs, the corresponding bit in the *mip* (Machine Interrupt Pending) register is set to indicate that the interrupt is pending. If the same bit is also set in the *mie* (Machine Interrupt Enable) register, it means that the interrupt is locally enabled. Only when both the *mip* and *mie* bits for a specific interrupt are set does the processor proceed to service that interrupt.

1.6.1 CLINT Controller

Core Local interrupt controller is responsible for handling the timer interrupts and software interrupts. There are three register which are memory mapped in the CLINT module. MSIP, MTIME and MTIMECMP. MSIP is of 32bit, MTIME and MTIMECMP register are of 64-bit each. These are asynchronous interrupts, as they are generated outside the core module. CLINT is a memory mapped device and needs to be written by the processor. Base address of these registers are mentioned in table 1.12.

CLINT Register	Address	Size
MSIP_BASE_ADDRESS	0x5F00_0000	32-bit
MTIME_BASE_ADDRESS	0x5F00_0004	64-bit
MTIMECMP_BASE_ADDRESS	0x5F00_000C	64-bit

Table 1.12: CLINT Base address

These register are configured by the software in the following manner:

Algorithm 1.2 Enable External Interrupt

```
/* Enable External interrupts */
1. MTVEC = <trap_handler_base_address>
2. MSTATUS = 0x0000_0008
3. MIE = 0x0000_0888
4. MTIME = 0x0001_8000
5. MTIMECMP = 0x0000_0888
6. MTIME = <Timer Interrupt>
```

MTIME stores the number of RTC clock values. If the RTC module is working on 32.768 Khz, then in 1sec it will store 32768 in *mtime* register. If we want an interrupt at every 3 sec, we need to initialise MTIMECMP with 3 left shifted by 15 (3*32768). This timer or external interrupt is converted into pulse and is given to the EXCEPTION UNIT. When these asynchronous expcetions occur, current PC is stored in the *mepc* and next PC is loaded as *mtvec + 4*offset*. This offset is fixed for machine Software, timer and external interrupts.

1.7 Memory Management Unit

The Sv32 scheme is the standard virtual memory translation mechanism for 32-bit RISC-V systems as shown in figure 1.11. It defines how virtual addresses (VAs) are translated to physical addresses (PAs) using a two-level page table system. This mechanism enables isolation and memory protection between different processes in a system. In Sv32, a 32-bit virtual address is divided into three parts:

- **VPN[1]** (bits 31–22): First level (root) page table index
- **VPN[0]** (bits 21–12): Second level page table index
- **Offset** (bits 11–0): Byte offset within the 4 KB page

Each page table contains 1024 (2^{10}) entries, and each Page Table Entry (PTE) is 32 bits. The page size is fixed at 4 KB.

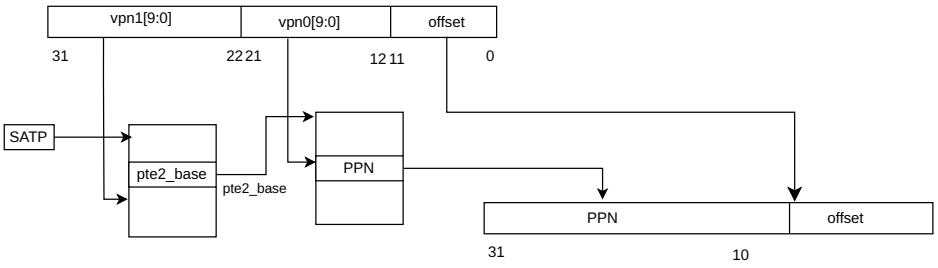


Figure 1.11: Sv32 memory translation

X	W	R	Meaning
0	0	0	Pointer to next Page Table
0	0	1	Read-Only page
0	1	0	*Reserved for future use*
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
1	1	0	Write-execute page
1	1	1	Read-write-execute page

Table 1.13: Encoding of PTE R/W/X fields

These translation is supported on both the side Instruction and Data. Virtual address is given by the core along with the translation request. If we get a TLB miss, then the page needs to be fetched from memory if it has valid PMP access.

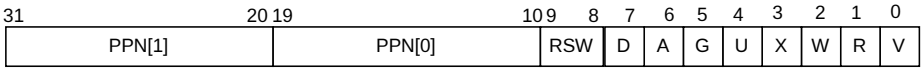


Figure 1.12: Sv32 page table entry

In the Sv32 virtual memory system, each Page Table Entry (PTE) follows the format shown in Figure 1.12 . The V bit (Valid) determines whether the PTE is considered active—if this bit is 0, the rest of the bits in the PTE are ignored by the hardware and can be repurposed by software. The R, W, and X bits represent access permissions, indicating whether the page is readable, writable, and executable, respectively. If all three permission bits are cleared (0), the PTE is treated as a pointer to a lower-level page table rather than a direct mapping to a physical page, identifying it as a non-leaf entry. Conversely, if any of the R, W, or X bits are set, the PTE is classified as a leaf entry and directly maps a virtual page to a physical page. The encoding of RWX bits is shown in figure 1.13.

Attempting to fetch an instruction from a page that does not have execute permissions raises a fetch **page-fault exception**. Attempting to execute a load or load-reserved instruction whose effective address lies within a page without read permissions raises a **load page-fault exception**. Attempting to execute a store, store-conditional, or AMO instruction whose effective address lies within a page without write permissions raises a **store page-fault exception.**

1.7.1 Page Table Walker

The two-level page table walker is a hardware mechanism that automates the translation process. When we get VPN (virtual page number) and translation request from the processor, corresponding translation is looked at the CAM (Content addresable memory), if there exsist a previous matching then the PPN is given out along with the **tlb_hit**

signal. If we encounter the miss, then the mapping needs to looked in the page table located in the memory. Hence we need to do the page table walk. Page table walker is shown in the figure 1.13.

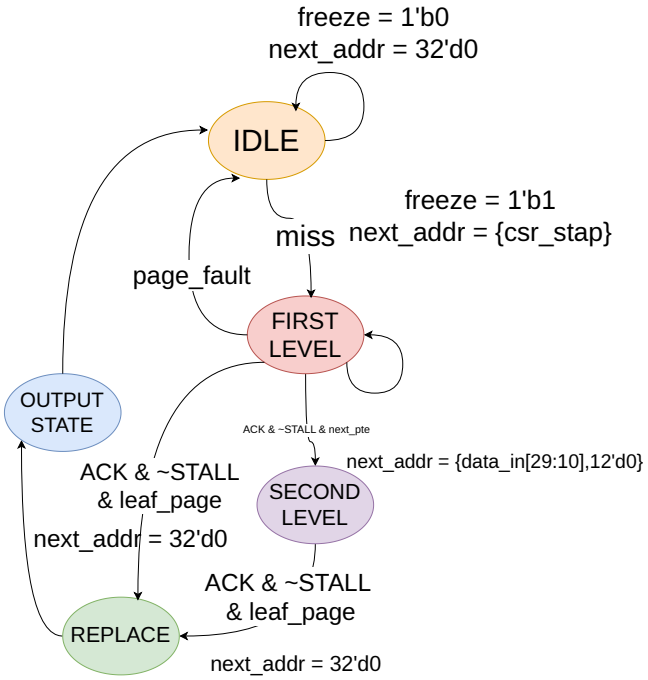


Figure 1.13: Page Table Walker

The translation process involves two stages, each corresponding to a level in the page table hierarchy.

Level 1 Translation

- Base Address Calculation: The base address of the level 1 page table is obtained from the satp register.
  - *satp.PPN* provides the base address of the level 1 page table.
  - The base address is shifted left by 12 bits to align with the page table entry size (4 KiB).
- Index Calculation: The index into the level 1 page table is derived from *vpn1*
  - vpn1[9:0] is used to index into the level 1 page table.
  - The address of the level 1 page table entry (PTE1) is calculated as: **pte1_address** = satp.PPN << 12 | vpn1[9:0] << 2
- PTE1 Access: The level 1 page table entry (PTE1) is accessed at the calculated address.

- PTE1 contains the base address of the corresponding level 2 page table.

Level 2 Translation

- Base Address Calculation: The base address of the level 2 page table is obtained from PTE1 if that is not the leaf entry.
  - The base address is shifted left by 12 bits to align with the page table entry size (4 KiB).
- Index Calculation: The index into the level 2 page table is derived from *vpn0*.
  - vpn0[9:0] is used to index into the level 2 page table.
  - The address of the level 2 page table entry (PTE2) is calculated as: **pte2_address** = pte1.PPN << 12 | vpn0[9:0] << 2
- PTE2 Access: The level 2 page table entry (PTE2) is accessed at the calculated address.
  - PTE2 contains the physical page number (PPN) and other metadata (e.g., permissions, accessed bit, dirty bit).

Final Physical Address Calculation

- PPN Extraction: The PPN is extracted from PTE2.
  - The PPN is combined with the offset to form the final physical address.
  - The final physical address is calculated as: **physical_address** = pte2.PPN << 12 | offset

  If either PTE1 or PTE2 is invalid (i.e., the V bit is 0), a page fault exception is raised. The exception handler can then take appropriate action, such as loading the missing page table entry or handling the fault. The permissions and access control are checked during the translation process:

1.7.2 VIPT cache

Since translation process requires fetching of page from the main memory and sometimes from DRAM (in case of OS), it can slow down the process of memory access. Hence to avoid that, we are using VIPT caches both for instruction and data memory. In VIPT caches, the index and block offset bits (12-bits in our case) are used to fetch cache line and parallely virtual_tag is given to the TLB for physical mapping (or physical tag). Once we get this *physical_tag*, it is compared against the *tag_out* from the cache memory to check if it's a hit or miss. The figure 1.14 shows how the VIPT caches are instantiated.

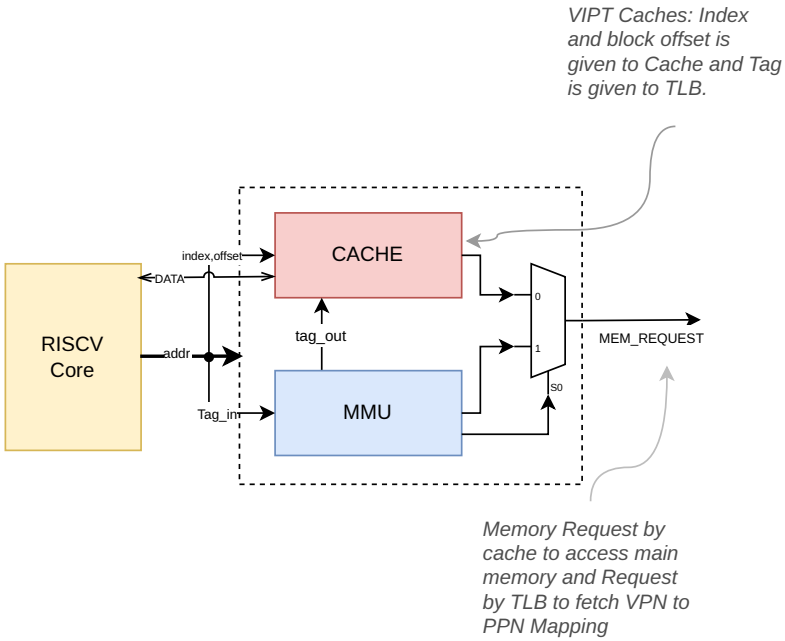


Figure 1.14: VIPT cache design

VIPT (Virtually Indexed, Physically Tagged) caches are a type of cache memory organization that combines the benefits of both virtual and physical caching strategies. In a VIPT cache, the cache is indexed using the virtual address, which allows for faster cache access since the translation from virtual to physical address is not required at cache lookup time. This can significantly reduce the latency associated with cache accesses, especially in systems where address translation is complex or involves multiple stages, such as in virtualized environments or systems with large page tables.

The cache lines in a VIPT cache are tagged with the physical address, which ensures that the cache coherence and consistency are maintained across different processes and address spaces. This tagging mechanism helps in avoiding conflicts and false sharing between different virtual addresses that may map to the same physical address, thereby improving the overall cache performance and reducing cache thrashing.

1.7.3 ITLB

The page table walker helps in the translation from virtual to physical address which then will be used for accessing the cache. The diagram 1.15 shows the complete ITLB design.

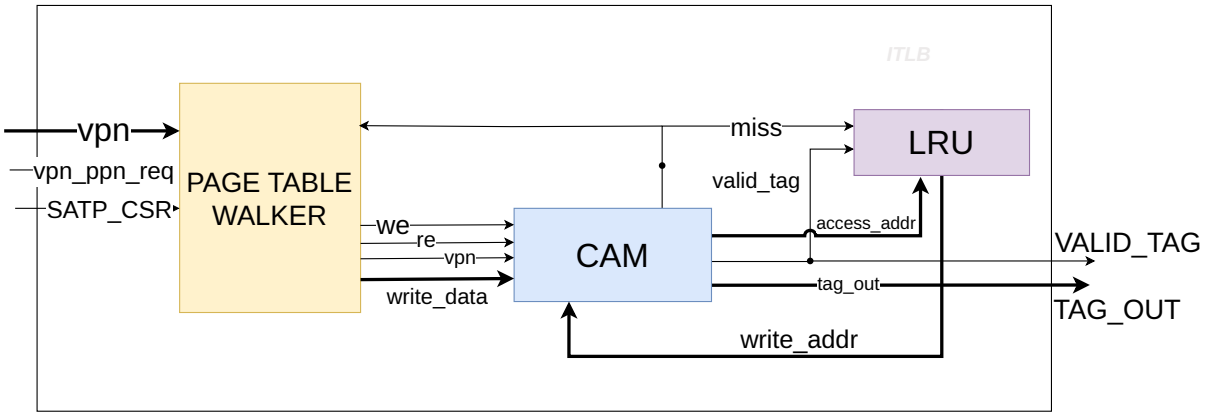


Figure 1.15: ITLB

Here if there is a miss from the CAM, the translation process starts and the processor remains stalled for that duration. Parallely <index+blk_offset > are used to access the cache line. When the translation completes we have the correponding physical address, and the mapping of VPN to PPN is written in the CAM which is also known as Translation Lookaside buffer (TLB). ***tag_ out*** and ***tag_ valid*** are given to the ICACHE which signifies the corresponding physical tag is valid.

There are total of 32 entries in the TLB, each entry of 52 bit (20 bits of VPN and 32 bit of PTE).

If all the entries of TLB are filled then the entry which is least recently accessed gets evicted with the help of LRU block.

1.7.4 DTLB

The structure of DTLB is similar to that of ITLB except that there will be 2 ports for accessing the data memory. One port is used by the LSU (Load store unit) for conventional Load/Store operation and another port is used by the decode stage for ATOMIC operation. In atomic operations, memory need to be read for flags or lock bit and it needs to be stored in the register before executing any other instructions. The figure 1.16 shows how the DTLB is integrated with the page table walker for fetching the virtual to physical mappings.

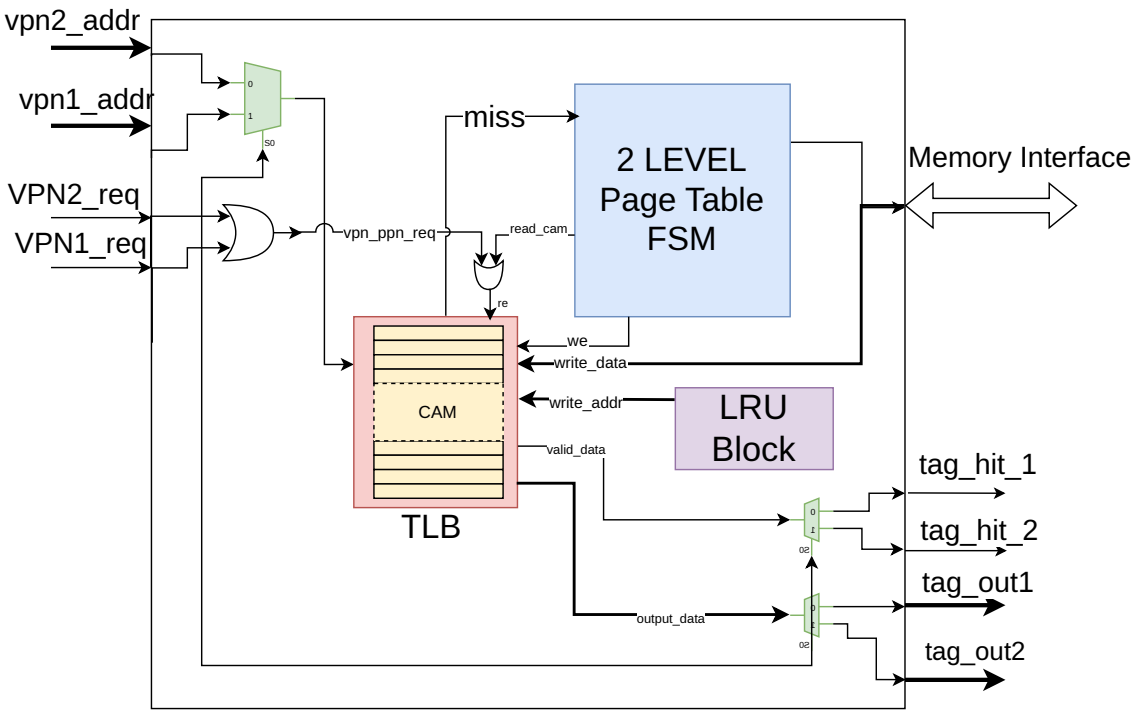


Figure 1.16: DTLB

1.8 ICACHE Memory Subsystem

The Instruction memory subsystem consists of **Instruction Cache module** (I-Cache and Instruction TLB (ITLB)) and the **Instruction Cache controller** as shown in the figure 1.17. Virtual Address is used for accessing the ICACHE. In our design we have Virtually indexed and Physically tagged cache i.e Virtual Index and Offset (PC_ADDR[11:0]) is used for accessing the index and the corresponding word in that Block. Parallely Virtual Tags are used to access the TLB that translates this Virtual Tag into Physical Tag. This Tag coming from the TLB is then comapared with the TAG coming from the CACHE Array to check whether it's a Cache hit or miss. If the Tag matches then it's a cache hit and corresponding Instruction is given to the processor pipeline.

The I-cache controller manages cache read and write. If it is a cache hit, the requested instruction is given to the fetch stage. If it is a cache miss, a write bus transaction is initiated on the memory bus unit. The page offset from the virtual address and the physical tag bits from the TLB form the address of the instruction line to be fetched and given to the memory interface. The figure 1.17 shows the Instruction cache subsystem.

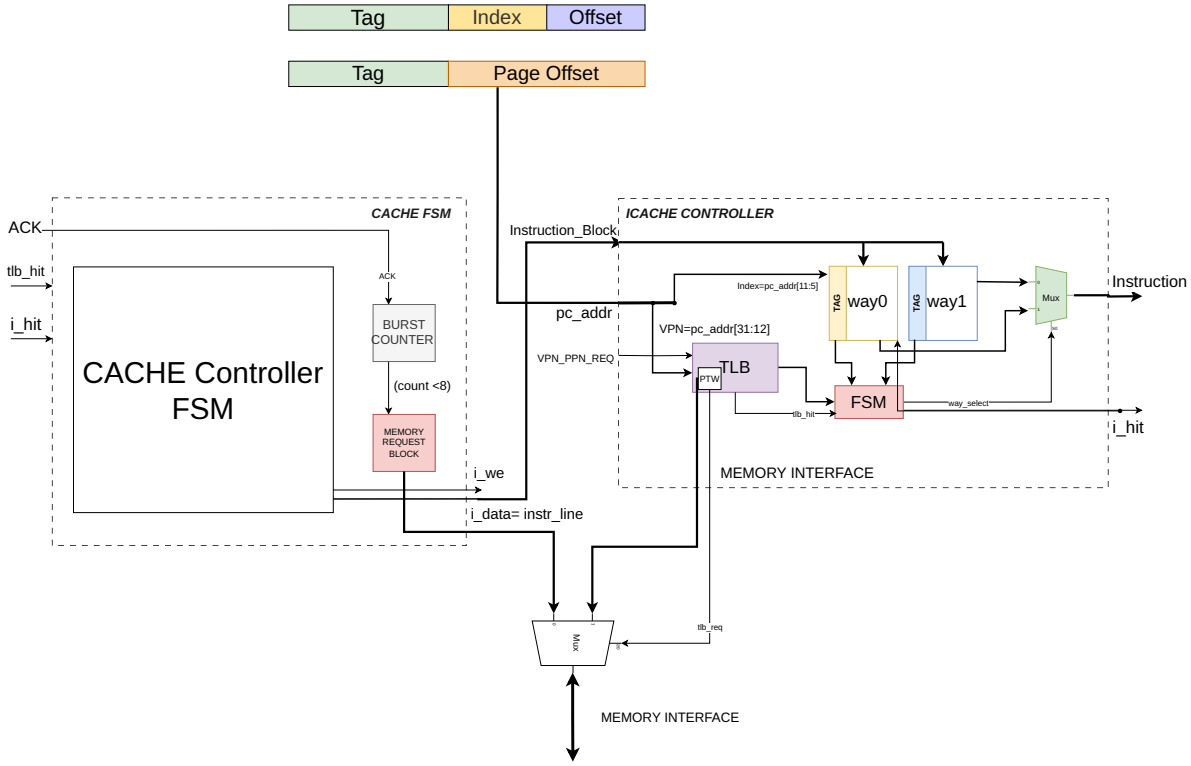


Figure 1.17: Instruction Cache Subsystem

The I-Cache generates instruction access fault and instruction page fault exceptions. Access fault exception is generated when the PMP and PMA checks on the physical address fail. Instruction page fault is generated by the TLB during page table walk when an invalid page table entry is encountered, or the page does not have execute permission. The exceptions generated by the I-Cache are carried along the pipeline.

1.8.1 Instruction Cache module

This module consists of ICACHE SRAM macros, ITLB and FSM, both of them work in parallel fashion as shown in the figure below.

The ICACHE is 8KB, 2-Way Set associative with block size of 32Bytes (8-Instruction). The I-TLB is a content Addressable memory with 32 entries.

There are 128 sets in the cache array, each of which has a 32Bytes of Block. Out of 32-bit of Virtual Address (PC_ADDR[31:0]), last 12 bits are used for accessing the Cache array of which upper 7 bits (PC_ADDR[11:5]) are used to address into cache array using Sets and last 5 bits (PC_ADDR[4:0]) are required to fetch the corresponsding byte in the Block. Since the instructions are aligned on 32-bit word boundary, last two bits of Block offset (PC_ADDR[1:0]) has to be 0. Configuration of ICACHE is given in table 1.14 and figure 1.18.

ICACHE Parameter	Value	Description
Total size	8KB	Total icache size is of 8KB
No of ways	2	Each way is of 4KB
Line size	32 Bytes	8 words per cache line
No of index	128	

Table 1.14: Icache organisation

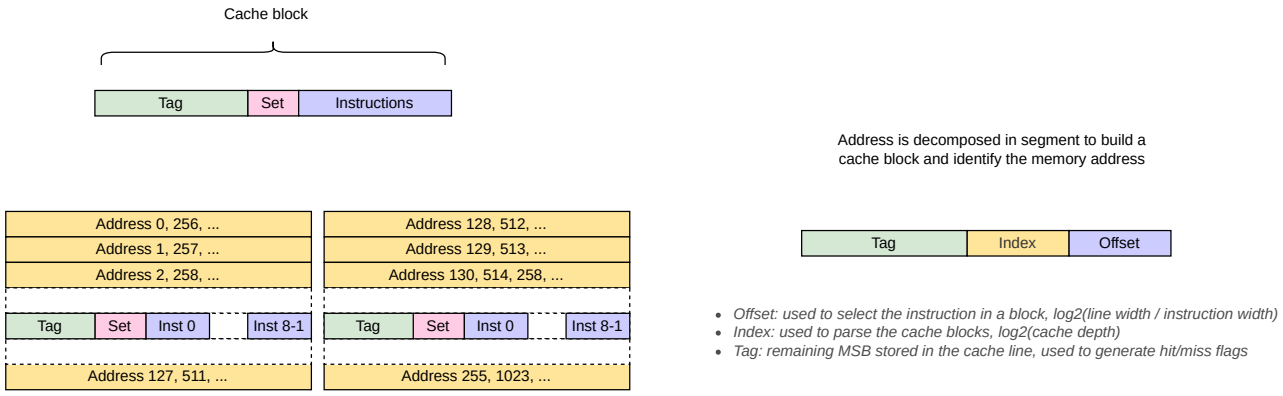


Figure 1.18: Icache Organisation

Cache line consists of 20-bits of Tag and 128-bits of Block. Tag array and Block array are designed separately, each of which is accessed by the Virtual index. When there's a TLB hit, in the next cycle Physical TAG from TLB is compared against the TAG coming out from the array. If they matches then we have the cache hit otherwise miss. If the cache hits then the corresponsing instruction is selected based on the block offset bits. If the cache misses, then corresponsding Instruction needs to be fetched from the Main memory. This process is taken care of by the *Cache controller* block. The figure 1.19 shows the structure of Instruction cache module.

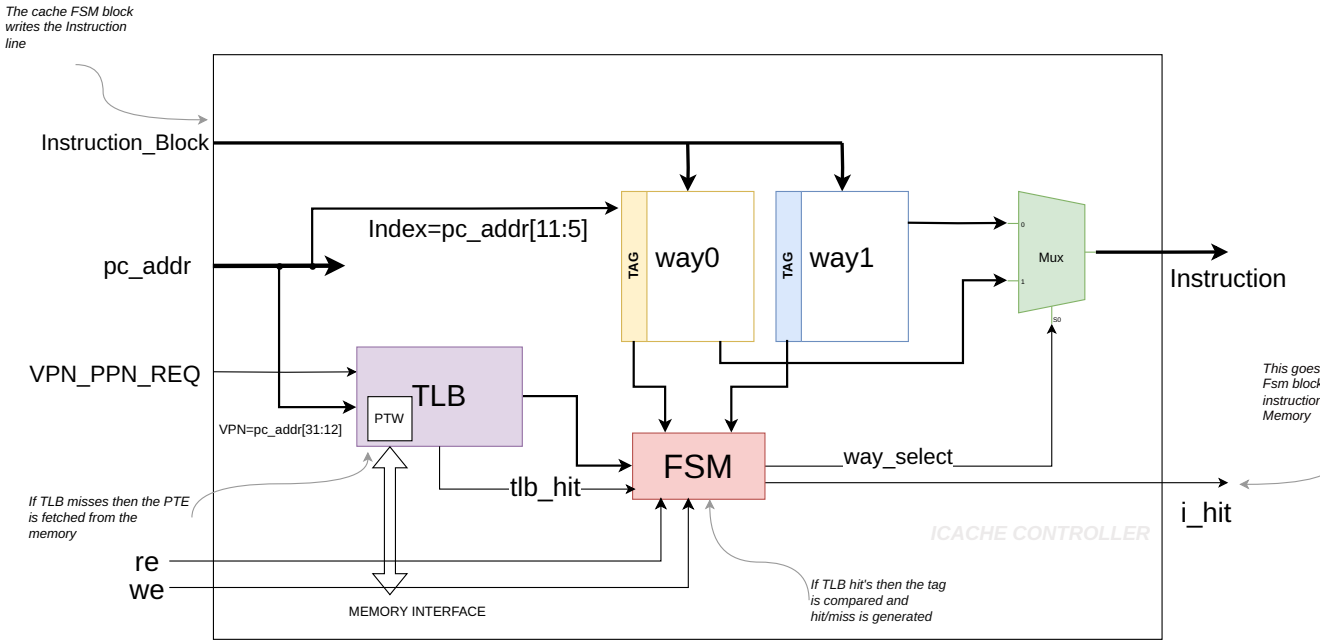


Figure 1.19: Instruction Cache module

1.8.1.1 Icache FSM

This FSM controls the writing of the instruction cache line in any one of they way and corresponding tag in the TAG macro during the cache filling stage.

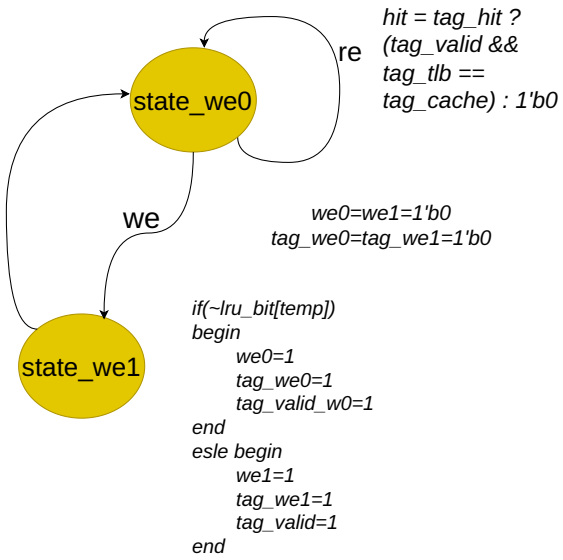


Figure 1.20: Icache FSM

Initially we are in the state_we0 state, when the re signals comes from the cache controller

indicating we have the read request from the processor. If we encounter tlb_hit, the tag is compared with the tag from the TAG array. If it matches it is a hit and the corresponding instruction is given out. If we encounter a miss, then we need to wait for the cache controller to service the miss. When the complete cache line (32Bytes) is received, we get the we signal and we move on to state_we1 state in next cycle. Then we decide on which way we need to write this instruction and tag line based on the LRU bit. Once we write the cache, it again goes to state_we0 in next cycle. Hit goes to 1 and the instructions are supplied back to the processor.

Instruction_miss_latency = 8cycle/instruction, Instruction_hit_latency = 1cycle/instruction

1.8.2 Cache FSM Module

When the TLB hit's and after comparision with Tag from the Tag Array, cache_hit signal is generated. If the cache_hit is 1'b1, then the corresponding Instruction is given out to the Pipeline Stage (IF). The below figure 1.21 shows the design of Icache controller module.

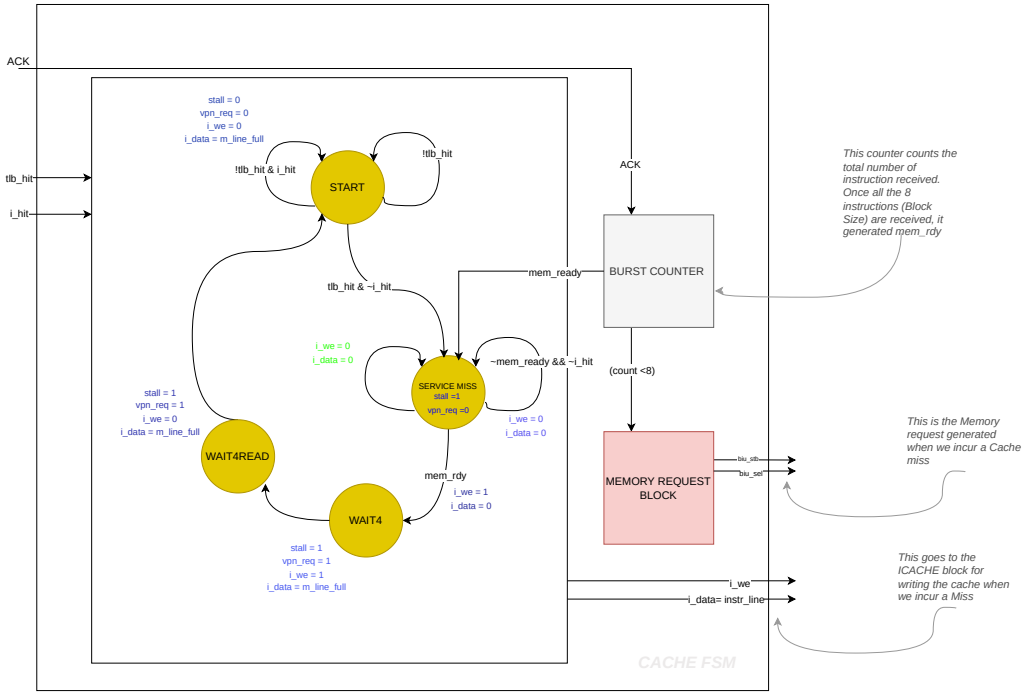


Figure 1.21: Icache controller module

The I-cache controller manages cache read and write. If it is a cache hit, the whole instruction line read is given to the fetch stage. If it is a cache miss, a write bus transaction is initiated based on memory request unit. If we have a cache miss, then we service the miss by fetching 8 consecutive instructions from the memory through burst and in next cycle we write the whole line in the cache array.

In WAIT4 and WAIT4READ state, *we* is high and Icache FSM writes the cache line in the corresponding way. In the next cycle we get a cache hit and the instruction is supplied back to the processor.

1.8.3 Icache protocol

Icache timing protocol is shown in figure 1.22 . The process is shown after the TLB hits and we have the PPN out from the TLB

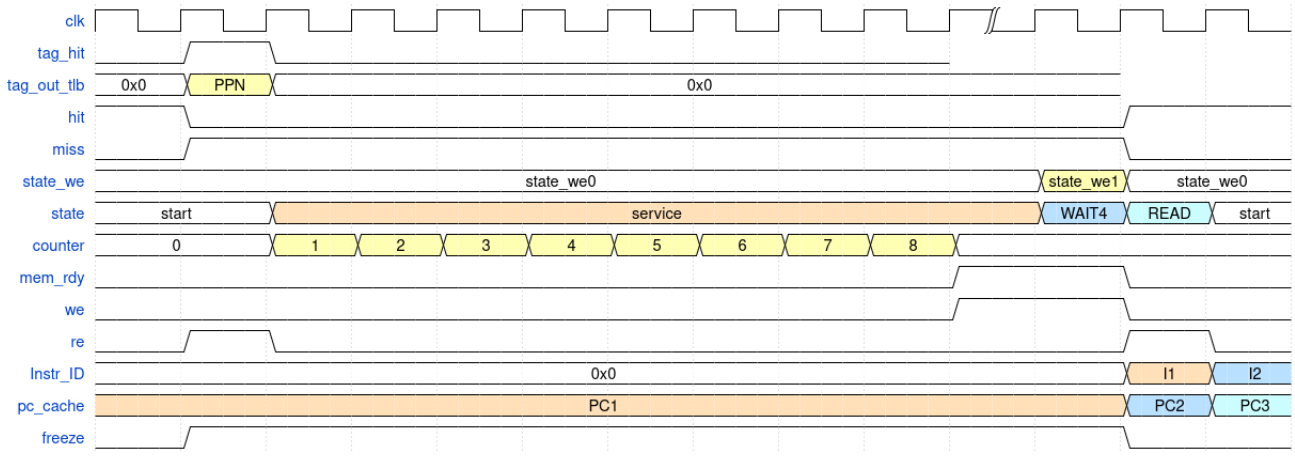


Figure 1.22: Icache protocol

The processor boots from the address 32'h0000_0000, which is the base address of the instruction memory (BOOT ROM) in our case.

When the first time processor sends the BASE_ADDRESS, there is a icache miss. Then the virtual address is given to the TLB block and physical address is given out with a tag_valid signal. In our case, since we have disabled the translation for ease of use, we'll have physical address = virtual address. When the *tag_hit* is 1'b1 and *hit_out* is 1'b0, then the state machine inside the **cachefsm** block goes to the service miss state(2'b01).

In the state SERVICE_MISS, the physical address is given to the instruction memory for fetching the instruction in burst. We have a cache line of 256 bits, that is 8 words (32-bits).

When the whole cache line is fetched, the cachefsm signals *we* to the icache controller for writing the cache line in to the memory. Physical tag along with the *tag_valid* is also written to the tag array memory. When *we* comes from the cachefsm, the FSM inside the icache controller state *state_we* goes to 2 (2'b10). In state 2, it cache line and tag is written on to the ICACE SRAM, and in next cycle it again comes to state 1.

Parallely, states of cachefsm are also moving. When the state of cachefsm is 2'b10, it raises the *vpn_to_ppn_req_3* signal, which goes to the icache controller and the state goes to 0.

This *vpn_to_ppn_req_3*, is used to read the tag again from the TLB and compare it with

the tag coming from the TAG SRAM ARRAY. It it matches, then it is a hit otherwise miss.

When the cache hits, it sends the *instruction[31:0]* to the pipeline unit using block select counter.

Now then the PC goes to 32'h0000_0020, the index increments and we encounter a cache miss, since the tag from the TLB(physical) doesnt matches with that from the memory. So signal i_hit goes down. Now the state in cachefsm again goes to SERVICE_MISS. At this moment, the tag_valid[VA] for the next upcoming way is written 1'b1 and cache again fetched the next line from the memory.

The figure 1.23 shows the timing waveform of the Icache module when it first encounters the cache miss.

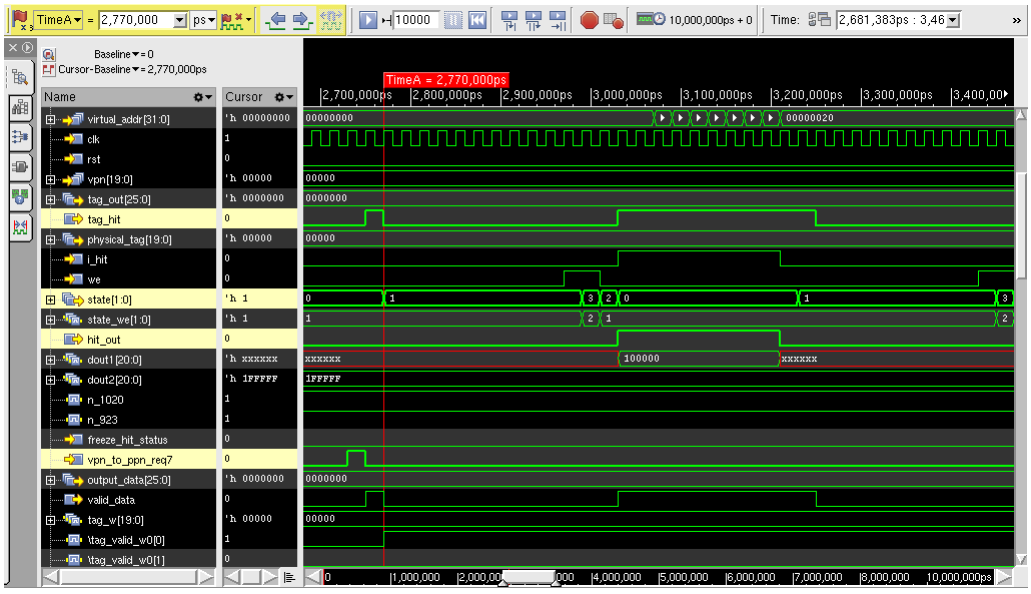


Figure 1.23: Initial icache miss

The valid bit for each cache line is designed as a register since the SRAMs are initialised to unknown state, and initial tag comparision results in an unknown state of a processor. Hence tag valid bits are designed as flop memories and tag array is the TSMC SRAM array.

1.9 DATA CACHE

1.9.1 Data Cache pipeline integration

The figure 1.24 shows the integration of DCACHE module with the Processor pipeline.

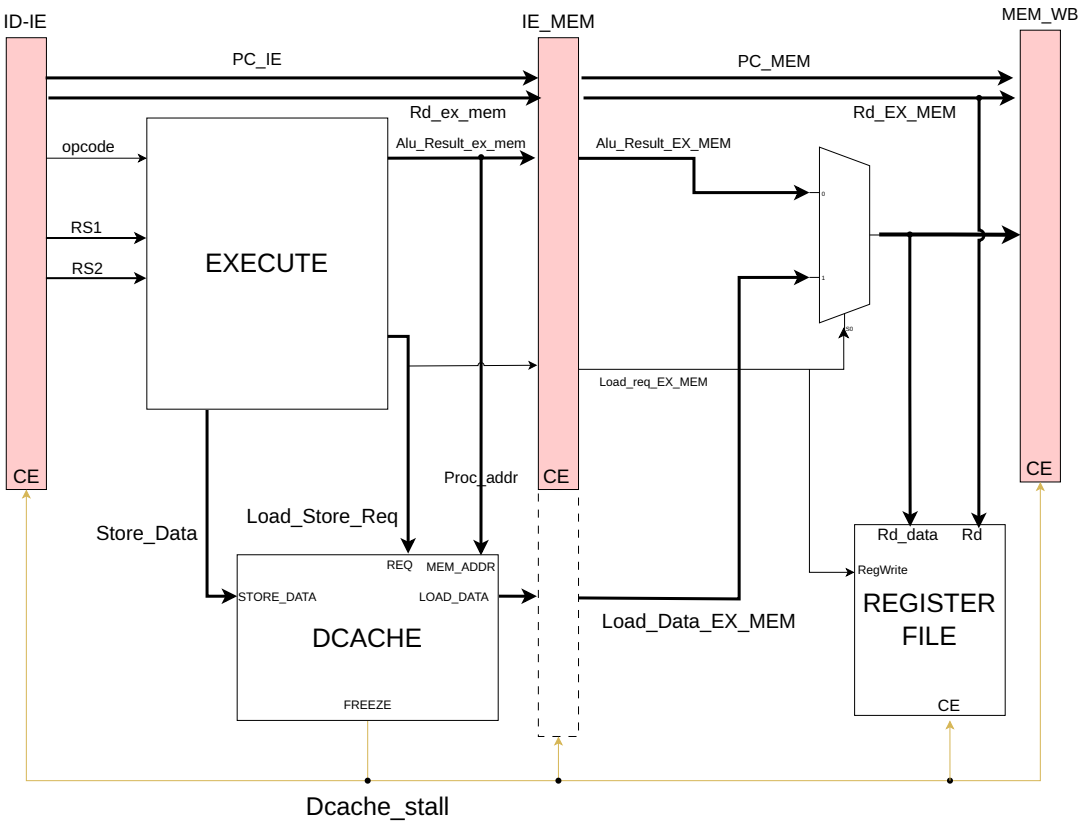


Figure 1.24: Dcache pipeline integration

When the load or store instruction enters the execute block, it generates the processor address and *Load_ Store_ req*. The request is directly given to the dcache so as to get the memory data in next cycle (if hit) and pipeline can work without any stall. If there is a cache miss in next cycle, *dcache_ stall* goes high in same cycle as miss and the pipeline will be stalled and the corresponding memory instruction enters the MEM stage. Miss will be serviced by fetching eight words from the Main memory and cache line is written when the whole line is available. On servicing the miss, the require data is supplied first (Criticle word first) to the processor. When the miss completes, the *dcache_ stall* is deasserted and normal pipeline operations resumes. The data is written in the register file in the write back stage.

DTLB is also integrated inside the DCACHE module and works in same fashion as ITLB. If we encounter a TLB_MISS, then the required mapping is fetched from the main memory with the help of 2 level page table walker. Once the TLB_HIT is 1, corresponding cache hit/miss is calculated based on tag comparision and valid bit.

1.9.2 Dcache organisation

The data memory subsystem consists of two modules, ***dcache_ fsm*** and ***dcache_ dpram.*** *Dcache_fsm* is responsible for generating the memory request if we encounter a cache

miss and deliver the Load data to the processor if we get a Load hit. Address must be aligned to 32-bit word bounday, hence Load/Store misaligned exception is raised if we get a misaligned address. DCACHE and DTLB works in parallel fashion (VIPT), where virtual index is used for indexing the memory and Physical tag from the TLB is used for checking the hit condition. The Dcache design is shows in figure 1.25

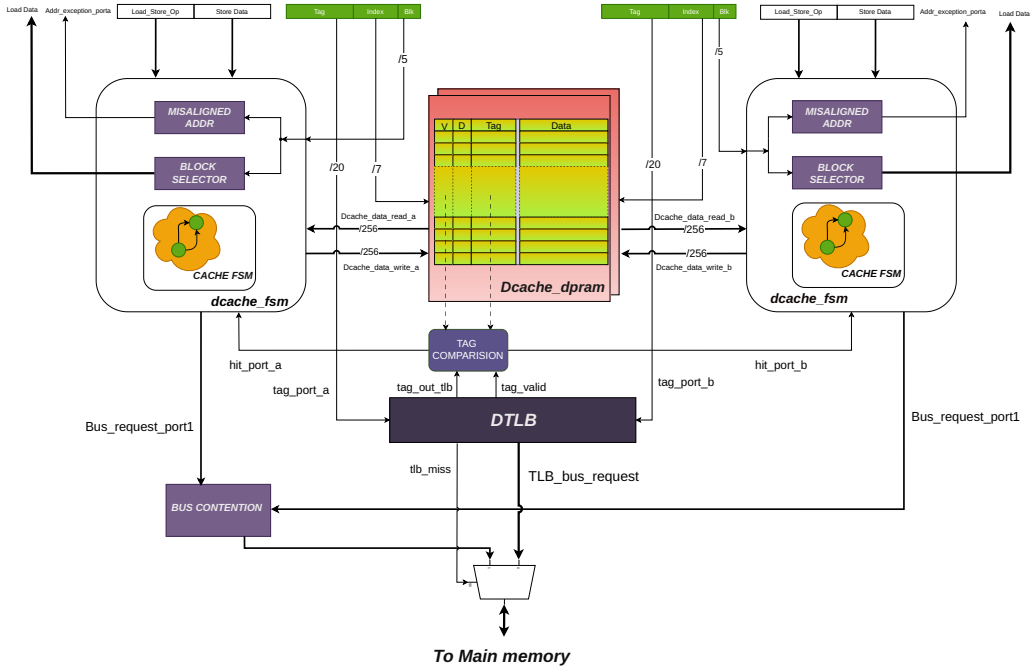


Figure 1.25: Dcache design

The D-Cache is 8KB, 2 Way set-associative cache with block size of 32-bytes. The D-TLB is a Content Addressable Memory with 32-entries. It supports simultaneous read and write requests and has two separate ports, one port is controlled by the Load-Store unit for conventional Load/Store or peripheral accesses and another port is for accessing the memory during AMO operations. Since both the request cannot occur simultaneously, we'll not experience a Structural hazard.

The D-Cache follows the write-back policy, i.e., the data is written only to the cache and not to the memory during stores. Only when this block needs to be replaced by a new block, the data is written back to memory. The D-cache supports word, half-word, and byte data type with aligned memory access. Data write is byte-level enabled. In the case of data read, the cache line of the cache way which is hit passes through the Block selector, and the selected bytes are given at output. In the case of data write, the input bytes are stored in the Write buffer and are written to the cache way which is hit.

The **dcache_fsm** handles the Load/Store/AMO request coming from the processor pipeline. It first performs the PMA checks in DTLB, whether correct R/W/X permissions are there. If the processor tries to write on a Read-Only page region or if we encounter a page fault, *Page fault exception* is raised. The physical address generated in the next cycle of request, along with the index and block is checked for the PMP permissions. If

the *dmem_ disallow* goes high, then the corresponding exception is raised. For example if we want to attempt a store in Read-only region, store fault is raised.

This Dcache supports the **cache flush** feature. Since it has write back caches, the store wont be visible to the main memory. Hence caches can be flushed using the custom CSR (Software). All the cache line with dirty bit gets written to the main memory.

TSMC Cut name	Instance Name	Size(words * bits)	Description
TSDN65LPLLA128X128M4F	ram_w0_1	128 X 128	Way0 Cache array-bank1
	ram_w0_2	128 X 128	Way0 Cache array-bank2
	ram_w1_1	128 X 128	Way1 Cache array-bank1
	ram_w1_2	128 X 128	Way1 Cache array-bank2
TSDN65LPLLA128X32M8F	tag_w0,tag_w1	128 X 32	Way0 tag Array ,Way1 tag Array

Table 1.15: TSMC Memory cuts

For the ASIC design, we need to replace all the BRAMs with the memory cuts from the foundary for that particular technology node. We are working on 65nm. The table 1.15 shows the memory cuts we are using in this design.

1.9.3 Data Cache FSM

Dcache controller FSM design is shown in the figure 1.26 .

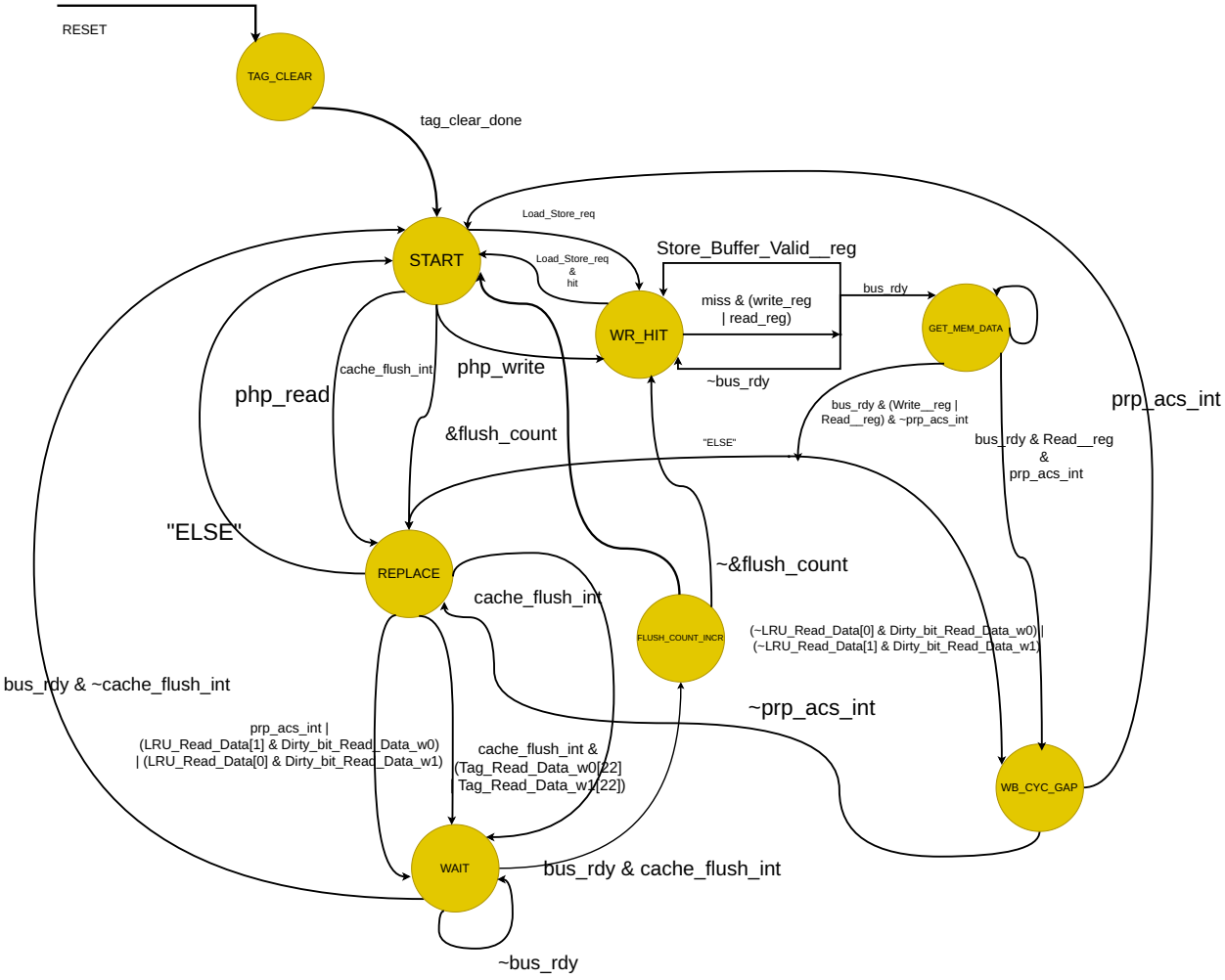


Figure 1.26: Dcache FSM

On reset, the initial state entered is **TAG_CLEAR** when the system is powered on or reset. In this state, all the cache line is invalidated before begining any cache operation. This is needed specifically in ASIC designs, where the intial state of memory is not defined (X). If we reset (Invalidate) all the cache line, then we can directly start the cache process.

1. **START**: The START state serves as the idle and decision-making state where the FSM waits for incoming memory access requests. If a ***Load_Store_req*** is detected, the controller goes to the WT_HIT state to complete the operation quickly. If *cache_flush_int* is asserted, then it goes to the **REPLACE** state.

2. **WT_HIT**: In the WT_HIT state, the cache controller checks whether there is a hit or a miss. If it is a read hit, the load data is written back to the processor, if it is a store operation it updates the appropriate cache line with the new data. The data may also be buffered using a store buffer if **Store_Buffer_Valid__reg** is set. This state allows for fast, low-latency completion of memory write operations without accessing main memory. If it is a cache miss, the state goes to the **GET_MEM_DATA**.

3. **GET_MEM_DATA**: The GET_MEM_DATA state is responsible for handling cache misses by initiating memory access to retrieve the requested data. The controller requests data from the memory subsystem and waits for the *bus_rdy* signal to indicate that the bus is ready to complete the transaction. If the bus is not ready, the FSM remains in **GET_MEM_DATA** state. Otherwise, it proceeds to **REPLACE** state to replace a cache line. If the current cache line is dirty, or if we have read request then it goes to **WB_CYC_GAP** to accomade that one cycle delay.

4. **REPLACE**: The REPLACE state is entered when a cache miss occurs and the existing cache line that will be replaced is dirty and needs to be written back. If there are no dirty lines, the cache line fetched is directly written on to the cache and goes to **START** state. The controller identifies the LRU line and evaluates its dirty status before replacement. If it is dirty, it goes to **WAIT** state for evicting the cache line back to the main memory. If *cache_flush_int* is asserted, it goes to state **WAIT** and writes the dirty line back to memory before bringing in the new data. This state ensures that no modified data is lost during the replacement process.

5. **WAIT**: The WAIT state acts as a holding state when the memory bus is not ready (~bus_rdy). In this state the cache line is written back to the main memory. This can happen in 2 scenarios, if the current cache line is dirty and it needs to be evicted, or if encounter *cache_flush_int,* then the dirty line has to written back to the memory. This ensures that the FSM does not proceed with memory operations until it can safely complete them. Once *bus_rdy* goes high, the FSM transitions back to **START** state to mark the end of cache process, or to **FLUSH_COUNT_INCR** if a cache flush is underway. This state helps coordinate timing between the cache controller and the memory system for reliable data transfers.

6. **WB_CYC_GAP:** The WB_CYC_GAP (Write-Back Cycle Gap) state introduces timing separation between memory write-back cycles. It allows the FSM to manage multiple pending write-back operations in a coordinated manner. This state can be used to align cache operations with bus cycles or to satisfy bus arbitration requirements. Once the write-back operation is appropriately spaced or completed, the FSM continues to the **REPLACE** state.

7. **FLUSH_COUNT_INCR**: The FLUSH_COUNT_INCR state is used during a cache flush operation to increment a counter that tracks how many cache lines have been flushed. This is necessary when a full cache flush is requested via Cache flush CSR. The controller loops through cache lines one by one, ensuring that dirty lines are written back and valid bits are cleared. Once all lines have been processed, the FSM transitions back to START.

8. **TAG_CLEAR**: The TAG_CLEAR state handles invalidation of cache tags, typically as part of a full cache flush. In this state, the controller clears the valid bits or tag fields to ensure all cached entries are marked as invalid. This is crucial for maintaining coherency and ensuring that stale data is not used in future operations.

If all tag entries are not yet cleared, the FSM moves to FLUSH_COUNT_INCR; otherwise, it returns to the START state when tag_clear_done is asserted.

1.9.4 Dcache miss protocol

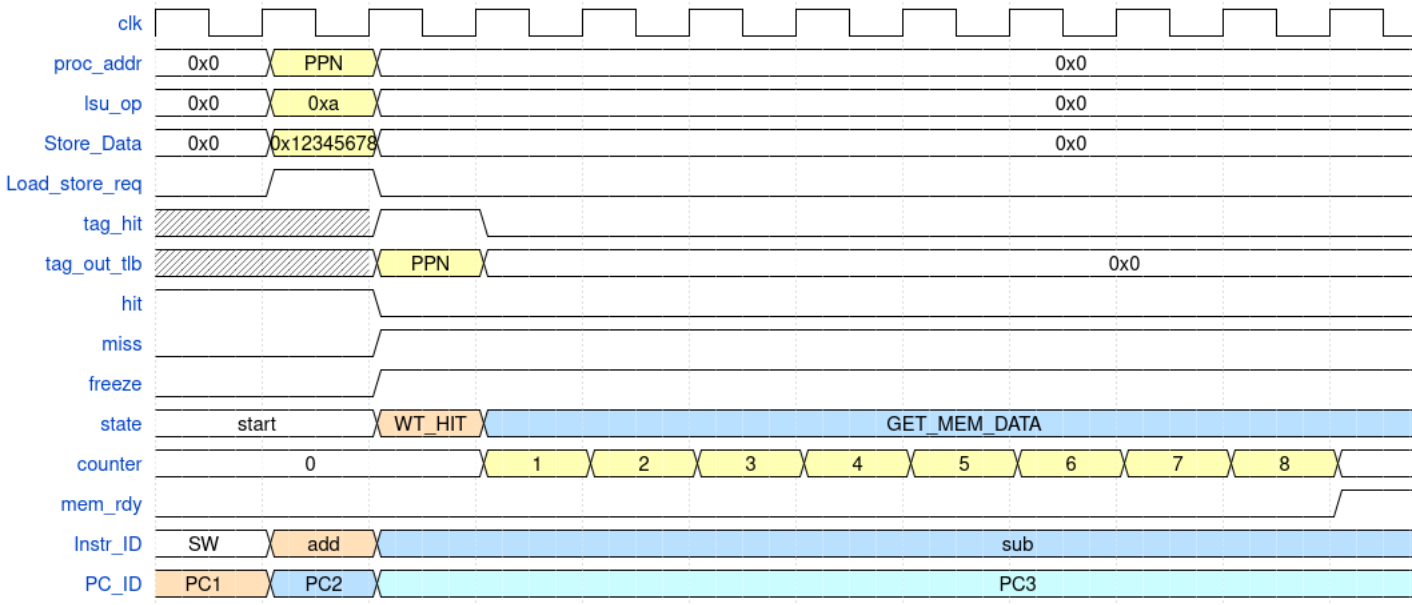


Figure 1.27: Dcache miss protocol

The figure 1.27 illustrates the behavior of the data cache (dcache) controller when servicing a cache miss. During the EXECUTE stage, the processor places the physical address (proc_addr), the store operation code (lsu_op), and the data to be written (Store_Data) onto the appropriate lines. In the subsequent cycle, the Physical Page Number (PPN) is available from the TLB, and a tag comparison is initiated. The FSM transitions into the WR_HIT state to perform a tag lookup. However, since there is no match (*tag_hit* is low), the controller detects a miss, and the pipeline is immediately stalled (freeze is asserted).

To service the miss, the FSM transitions to the GET_MEM_DATA state, where it begins retrieving the missing cache line from main memory. A counter increments with each word fetched, as shown in the waveform. Once the entire line is retrieved and *mem_rdy* goes high again, the FSM enters the REPLACE state to write the fetched cache line into the cache. After this, the FSM returns to the START state, allowing the pipeline to resume execution.

In the case of a store hit, the write operation is directed to the store buffer, allowing the processor to proceed without stalling. However, in a store miss, the pipeline stalls, and the FSM follows the same miss-handling sequence described above. Notably, the critical word (the one initially requested) is forwarded to the processor as soon as it is fetched from memory, rather than waiting for the entire line, improving perceived memory latency.

1.9.5 Dcache hit protocol

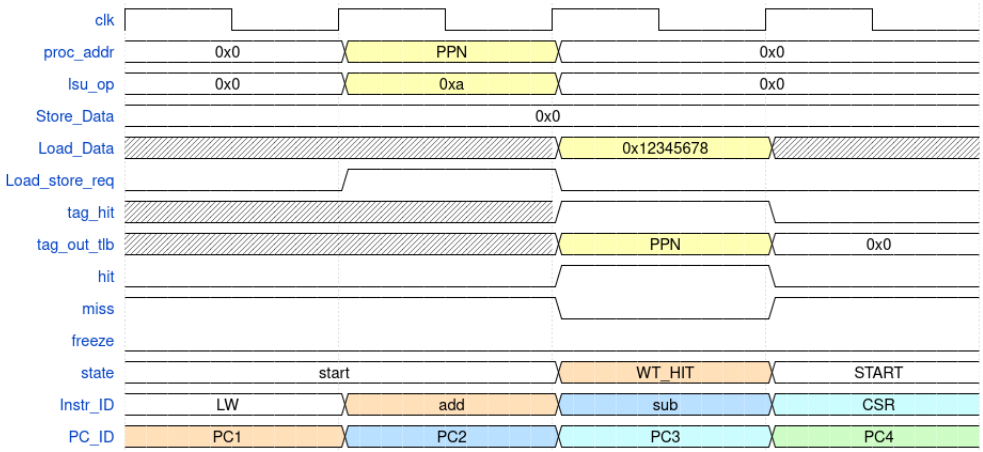


Figure 1.28: Dcache hit protocol

The waveform 1.28 depicts the behavior of the data cache (dcache) controller during a read hit. At the beginning of the EXECUTE stage, the processor asserts the physical address (*proc_ addr*), load/store operation code (*lsu_ op*), and initiates a memory access through the *Load_ store_ req* signal. In the following cycle, the Physical Page Number (PPN) is available via the TLB (*tag_ out_ tlb*), and a tag comparison is performed using the tag from the tag array.

Since the tag matches, a hit is registered (*tag_ hit* is high and *miss* is low), and the FSM transitions from the start state to WR_HIT. During this state, the requested data is directly fetched from the cache and placed on the Load_Data line (e.g., 0x12345678), enabling fast, single-cycle data retrieval. The pipeline proceeds without any stall (freeze remains low), and the FSM returns to the START state in the next cycle to handle subsequent memory requests.

This efficient hit-handling mechanism enables the load instruction (LW) to retire quickly while allowing the pipeline to continue executing subsequent instructions (add, sub, CSR) without delay, thereby maximizing instruction throughput.

1.9.6 Cache flush mechanism

The layout of DCACHE as shown in the figure 1.29 . LRU and Dirty bit memory is separate from the DCACHE and TAG array. Actual tag is of 20 bits, along with Valid bit (22) and Write bit (21).

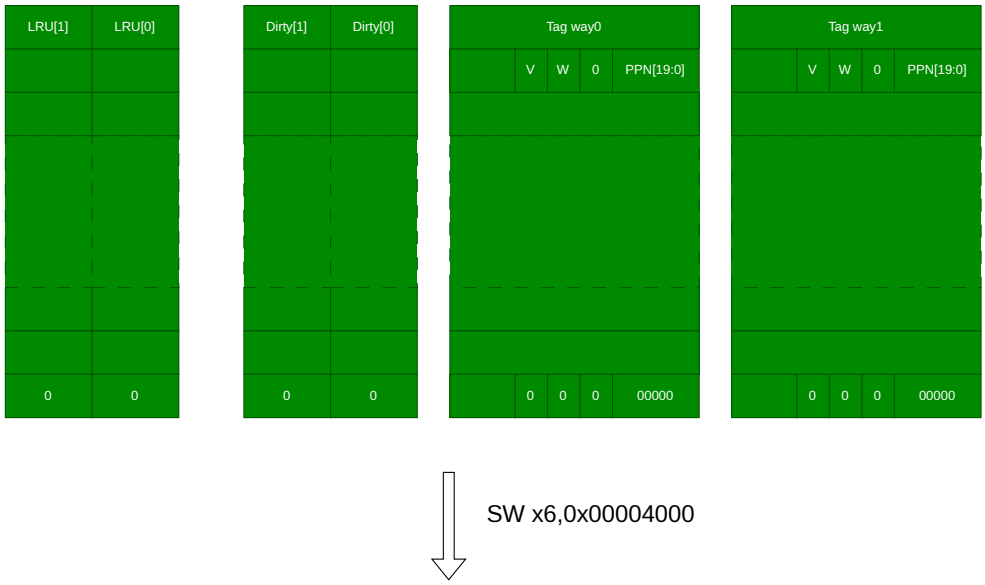


Figure 1.29: Dcache layout

LRU bit 0 means that cache line is next to be filled it misses on that index. Since it is a write-back cache, dirty bit goes high whenever the cache line is different from the main memory and it signifies that before evicting it, we need to store that line back to the main memory. Valid bit signifies that the corresponding cache line is valid, and write bit signifies that some store operation is done on that line.

Cache line will be flushed automatically to the main memory when some new cache line needs to be placed at the same location if it is dirty. But if the software wants to flush a cache line without waiting on it for the next line, the cache flush feature can be used. This is controlled by the software using the CSR register (*Cache Flush*). It can be used by the OS or software to if the writes needs to be visible on to the main memory even before accessing new line.

Cache flush CSR is located at 0x400 address of CSR memory map. LSB bit of that CSR indicates whether cache flush has to take place or not.

To understand this lets take a scenario as shown in the figure 1.30 . Lets say the processor executes following store instruction sw x6, 0x0000_4000. Hence it needs to write the data at x6 to the location 0x0000_4000. Since this cache line is currently not there, there's a miss and the whole cache line will be brought to the dcache array and correspinsing tag (0x00004), along with the valid bit and write bit will be stored in the TAG array of the selected way. Since LRU[0] is 0, the way0 is selected and it is written 1. The dirty bit is also written to 1 as shown in the figure 1.30 .

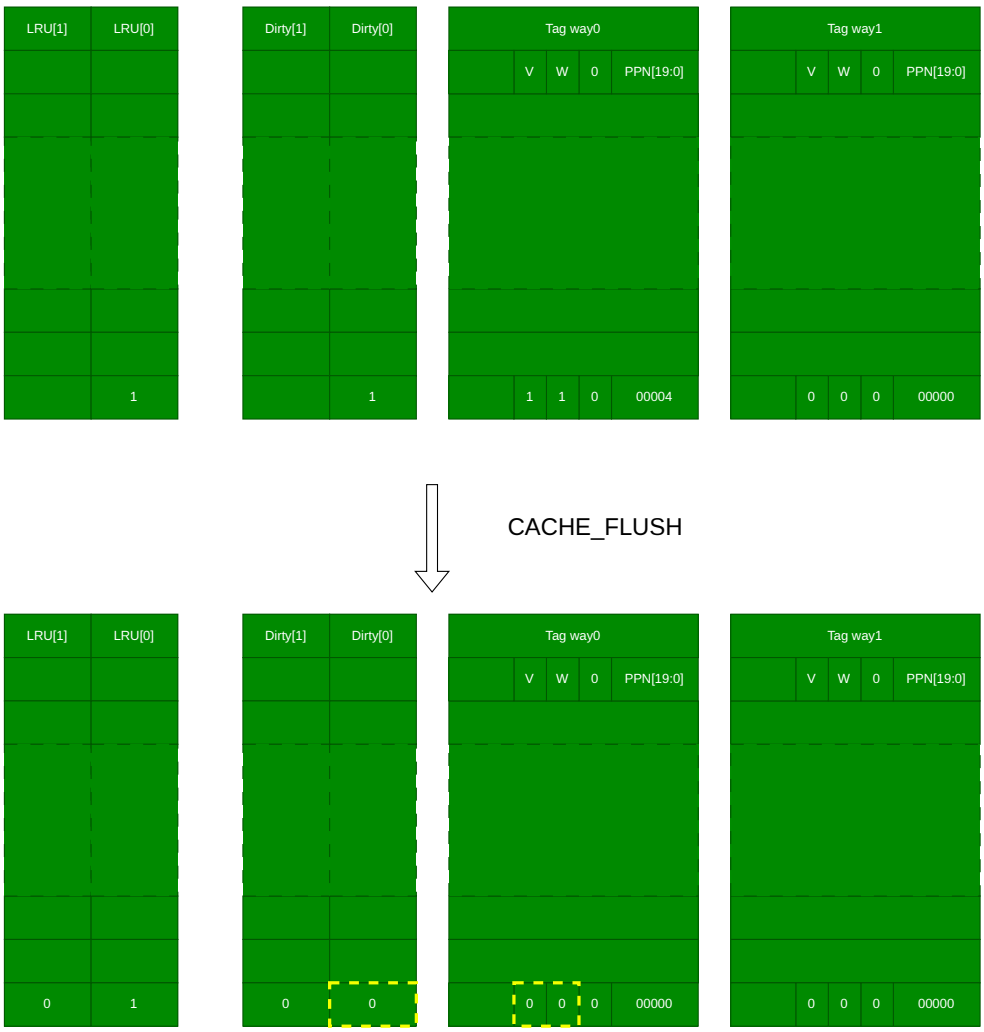


Figure 1.30: Cache flush mechanism

When the cache flushes, following thing occurs:

1. Reset the tag line (Write valid =0, write =0)
2. Reset the dirty bit
3. Reset LRU bit since that line is flushed.

The whole cache line is written on to the memory.

When switching between processes or threads, the cache may hold data relevant to the previous context. Flushing ensures that the new process doesn't see or interfere with stale cache data from the previous one — this is important for security and correctness. Also in multi-core RISC systems, each core has its own cache. To ensure coherency across cores, one core may need to flush (write back) or invalidate cache lines so others see the updated data.

1.10 Memory Protection Unit

1.10.1 MPU pipeline integration

The figure 1.31 shows the integration of MPU with the processor pipeline.

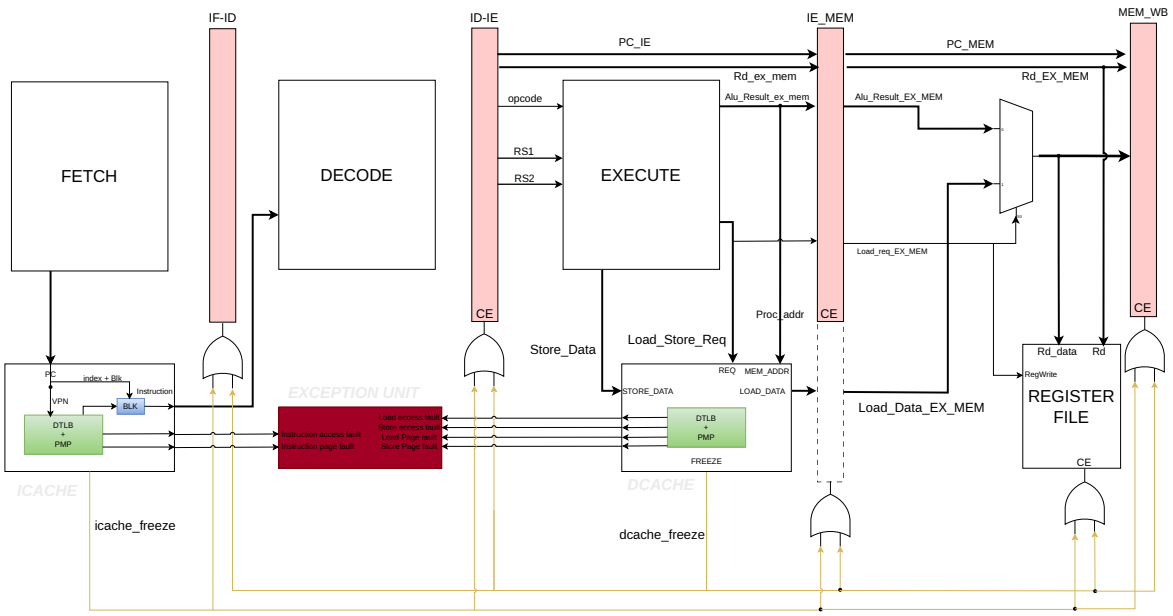


Figure 1.31: MPU pipeline integration

The PMP (Physical Memory Protection) block plays a critical role in enforcing memory access permissions in a RISC-V processor pipeline by integrating tightly with both instruction and data access paths. As shown in the figure 1.31, the PMP is connected in parallel with the DTLB (Data Translation Lookaside Buffer) and ITLB (Instruction TLB) and performs permission checks immediately after address translation. When a virtual address is translated to a physical address, the PMP verifies whether the access type (instruction fetch, load, or store) is allowed for that physical address range based on the currently active PMP configuration.

In the instruction fetch path, the PMP works alongside the ITLB. If the instruction access does not meet the PMP permissions, an instruction access fault or instruction page fault is raised and directed to the exception unit. Similarly, on the data access path, after address translation by the DTLB, the PMP checks whether a load or store is permitted. Violations trigger exceptions like load/store access faults or page faults. These exceptions are routed to the exception unit, which stalls the pipeline by asserting control signals like *icache_freeze* or *dcache_freeze* to prevent further instruction or data processing until the fault is resolved.

The PMP regions are configured during booting and can be fixed for a specific application. By default, the boot region is assigned read and execute permissions to allow execution of boot code, after which additional permissions can be configured for other memory

regions. Since this processor design does not support USER and SUPERVISOR modes, all PMP permissions are applied in machine mode, ensuring that protection policies are enforced even at the highest privilege level.

Suppose if we are at the user privilege level, then some of the memory regions would not be allowed for the user program to access. Then a user can switch to machine mode privilege level and can change the configuration of these PMP CSRs. Accessing these CSRs from user or supervisiory level will result in illegal instruction exception.

1.10.2 MPU block diagram

The figure 1.32 shows the Memory Protection Unit design.

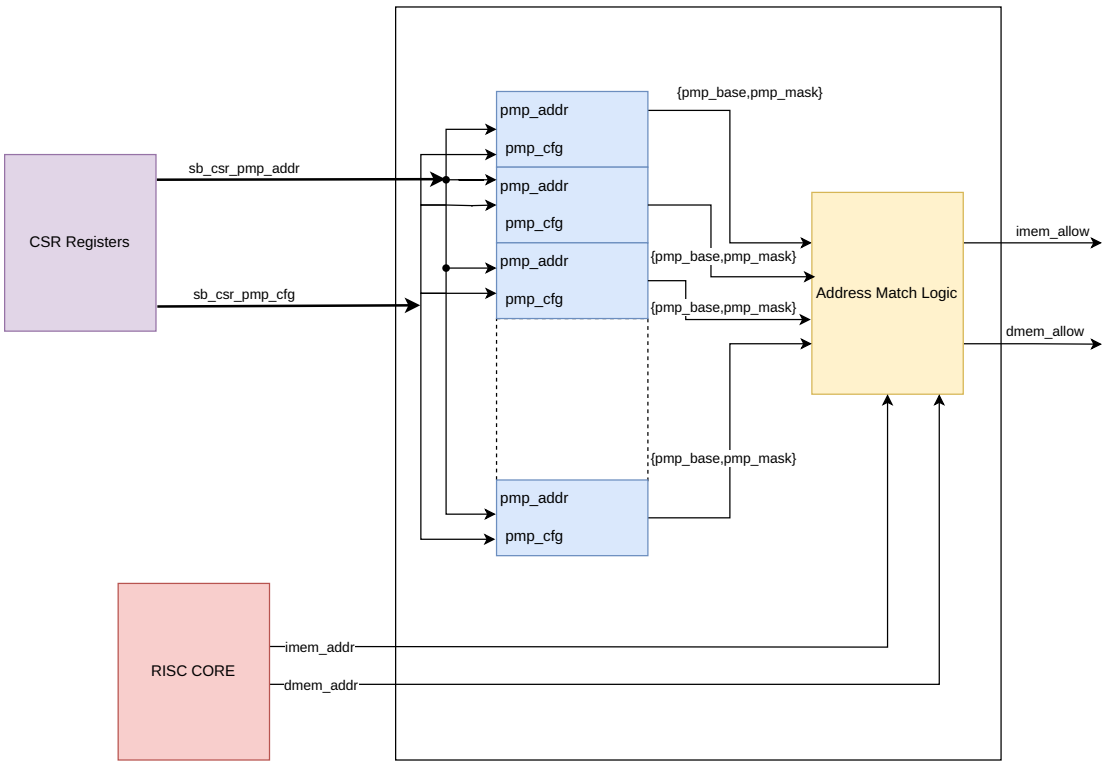


Figure 1.32: MPU RTL design

The internal block diagram of the MPU (Memory Protection Unit), based on the RISC-V Privileged Specification[**?**] , demonstrates how the system enforces memory access control through programmable physical memory protection (PMP) regions. At the heart of the MPU are the PMP address and configuration registers, which are configured via the CSR (Control and Status Register) interface. These registers store the base addresses (pmp_base) and corresponding masks (pmp_mask) that define memory regions, along with permission attributes such as read, write, and execute, encoded in pmp_cfg.

The MPU receives addresses from both instruction memory ($imem\_addr$) and data memory ($dmem\_addr$) paths. These addresses are passed to address match logic blocks, where they are compared against each defined PMP region using the configured base and mask values. The result of this matching determines whether access is permitted based on the region's permissions. For instruction accesses, the result is reflected in $imem\_allow$, and for data accesses, in $dmem\_allow$.

The PMP configuration and address registers ($sb\_csr\_pmp\_cfg$, $sb\_csr\_pmp\_addr$) can be updated by machine-mode software, allowing dynamic configuration of memory regions. Since only machine mode is supported in this implementation (no user or supervisor mode), all access checks apply uniformly in this highest privilege level. The

MPU thus acts as a gatekeeper that filters memory operations before they reach memory, ensuring only authorized instruction fetches and data accesses are allowed, as mandated by the RISC-V privilege architecture.

1.10.3 PMP CSR registers

The figure 1.33 shows the CSR registers used for configuring the memory regions.

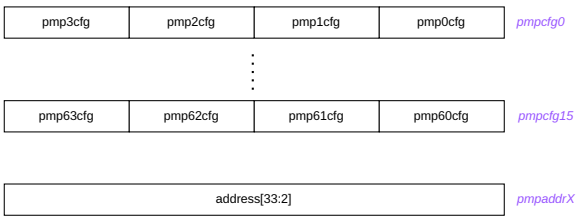


Figure 1.33: PMP CSR registers

The Physical Memory Protection (PMP) feature in RISC-V is configured through a set of dedicated CSR (Control and Status Register) registers, primarily consisting of *pmpcfg* and *pmpaddr* registers. The pmpcfg registers define the access permissions and address matching modes for each PMP entry. Each pmpcfg register (e.g., *pmpcfg0*, *pmpcfg1*, ..., up to *pmpcfg15* in RV32) is 8 bits per PMP entry and packs four entries per 32-bit register. Each 8-bit configuration field includes flags for read (R), write (W), and execute (X) permissions, as well as an address-matching mode (A) and a lock bit (L) that can prevent modification until the next reset.

Corresponding to each *pmpcfg* entry is a pmpaddr register (e.g., *pmpaddr0*, *pmpaddr1*, etc.), which defines the base address of the protected region. In RV32 systems, these registers use bits [33:2] of the address space, allowing memory regions to be defined in granularity of 4 bytes. The combination of these registers enables fine-grained control over memory access, making it possible to define secure execution regions, restrict access to critical peripherals, and isolate tasks. These registers are only writable in machine mode and are essential for enforcing the memory protection model defined in the RISC-V privileged architecture.

1.10.4 Address match logic

The figure 1.34 shows the address match mechanism used for allowing access to the valid region of physical memory for that particular thread.

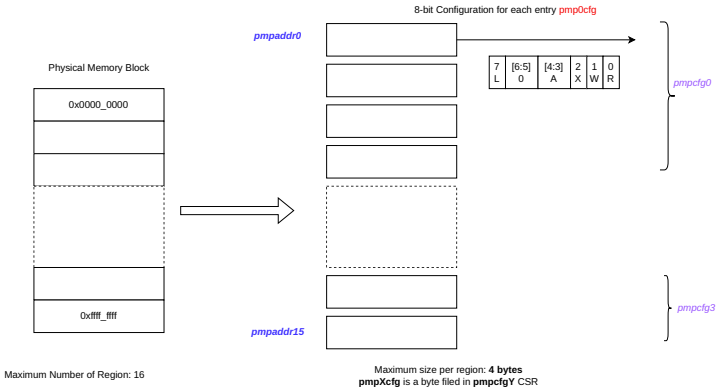


Figure 1.34: Address Match mechanism

In the RISC-V PMP (Physical Memory Protection) mechanism, memory regions are identified and access permissions are enforced using the *pmpaddrX* and *pmpcfgY* CSR registers. In our specific implementation, the physical memory is divided into 16 distinct regions, allowing fine-grained access control. Each region is associated with a corresponding *pmpaddrX* register, which defines the region's address boundaries. Permissions for each region are specified using 8-bit entries within *pmpcfgY* registers, where each pmpcfg can accommodate up to four 8-bit configuration fields, thus covering a total of 16 regions across *pmpcfg0* to *pmpcfg3*.

Each 8-bit configuration field includes bits for Read (R), Write (W), Execute (X) permissions, an Address matching mode (A), and a Lock bit (L). The A field plays a crucial role in defining the address range matching scheme. When A = 00, the region is disabled—no access is allowed. When A = 01, the Top of Range (TOR) mode is enabled, where the region starts at the previous PMP address and ends at the current pmpaddrX. This is typically used to define the first region. When A = 10, the region covers a fixed 4-byte range at the address specified by pmpaddrX. Finally, when A = 11, the NAPOT (Naturally Aligned Power-Of-Two) mode is used, allowing flexible region sizes that are aligned and sized to powers of two (e.g., 8, 16, 32 bytes, etc.), offering scalable and efficient region mapping.

This setup allows machine-mode software to configure and lock down access to different parts of memory at boot time, enforcing strict isolation and security guarantees in systems without user or supervisor modes.

1.10.5 Sample PMP configuration

To configure a region using the Physical Memory Protection (PMP) unit, both the base address and the size of the region must be specified. RISC-V supports two primary addressing schemes for PMP entries: TOR (Top of Range) and NAPOT (Naturally Aligned Power of Two). In the TOR scheme, the address specified in a PMP entry defines the upper bound of the protected region, while the lower bound is defined by the previous PMP entry. This means that access is permitted from the address in the previous entry

up to, but not including, the current entry's address. For the very first region, if TOR is used, the start address is implicitly taken as 0x0000_0000. For configuring intermediate or more complex memory regions, the NAPOT scheme is preferred. NAPOT allows specifying memory regions that are naturally aligned and sized as a power of two, using a special encoding format in the address field to represent both the base address and the region size simultaneously. This encoding simplifies memory protection for commonly aligned and sized regions, and enables more compact representation of memory ranges. The encoding is given in the table 1.16.

Base address	Region size	pmpaddrX value
addr[31:0]	8B	addr[33:2] | 1'b0
addr[31:0]	32B	addr[33:2] | 3'b011
addr[31:0]	4KB	addr[33:2] | 10'b01_1111_1111
addr[31:0]	16KB	addr[33:2] | 12'b0111_1111_1111

Table 1.16: Address and size encoding for NAPOT scheme

The position of 0 (LSZB) determines the size for the particular region. We can use following formula for the same:

$$RegionSize = 2^{(LSZB+3)}$$

Hence if we have region of 16KB, then LSZB has to be 11. It has to be ored with the base address right sifted by 2.

Below section gives few of the memory region configurations for this core, and if further section needs to be added, it can be added in the same way.

1.10.6 Boot ROM configuration

The figure 1.35 shows the Boot ROM region.

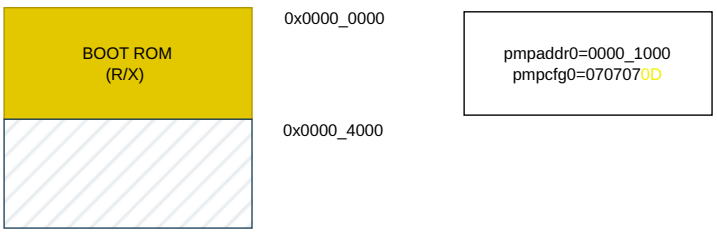


Figure 1.35: Boot ROM region

Since it is the very first region, we may use the TOR (Top of Range) scheme. Hence we right shift the top address (0x0000_1000) and store it in PMPADDR0 region. Since it has the RX attributes, the value of pmpcfg0 is 0x0D.

1.10.7 SRAM Configuration

The figure 1.36 shows the SRAM region.

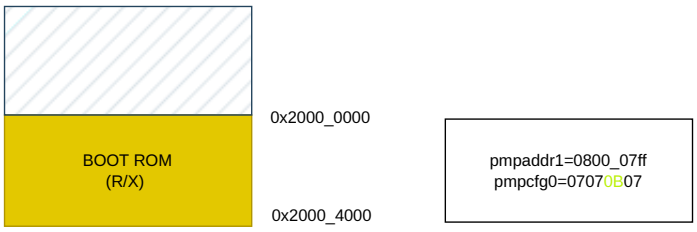


Figure 1.36: SRAM region

We will be using NAPOT (Natually aligned power of 2) addressing scheme for this region. So we right shift the base address by 2 (0x0800_0000). Since the size of this region is 16KB, hence the value of LSZB is 11.

Hence the final value of PMPADDR1 is 0x0800_07FF. Since it has the attribute of R/W, the value of pmpcfg register will be 0x0707**0B**07. Following assembly command 1.3 is used to write both the configuration.

Algorithm 1.3 PMP configuration for SRAM region

```
.equiv PMP_REGION0, 0x00004000
.equiv PMP_REGION1, 0x20000000
li t0,PMPADDR0
srli t0, t0, 2
csrw pmpaddr0, t0
li t0, PMPADDR1
srli t0, t0, 2
csrw pmpaddr1, t0
li t0, 0x07070B0D
csrw pmpcfg0, t0
```

1.11 Memory Map

The memory map of a processor as shown in table 1.17 defines how different regions of memory are organized and accessed within the system's address space. It serves as a blueprint that assigns specific address ranges to various functional units such as RAM, ROM, peripheral devices, memory-mapped I/O, and system control registers. In systems with virtual memory support, the memory map also distinguishes between user and privileged address spaces, allowing for isolation and protection. The memory map ensures that software knows where to load programs, where to store data, and how to interact with hardware. It also plays a critical role in defining areas used for page tables, interrupt vectors, and boot code.

Address Range	Name	R/W/X	SIZE
0x0000_0000 - 0x0000_4000	BOOT ROM	RX	16KB
0x2000_0000 - 0x2000_4000	SRAM	RW	16KB
0x5F00_0000 - 0x5F00_4000	PERIPHERAL	RW	16KB
0x5100_0000 - 0x5100_4000	DEBUG	RX	16KB
0xF000_0000 - 0xF000_4000	PAGE TABLE	RWX	16KB
0xFF00_0000 - 0xFF00_4000	INTERRUPT HANDLER	RWX	16KB

Table 1.17: Memory Map

A well-structured memory map enables efficient access, simplifies system software design like operating systems and drivers, and supports features like memory protection, caching control, and device-specific optimizations. Visual representation of the physical memory region is given in figure 1.37.

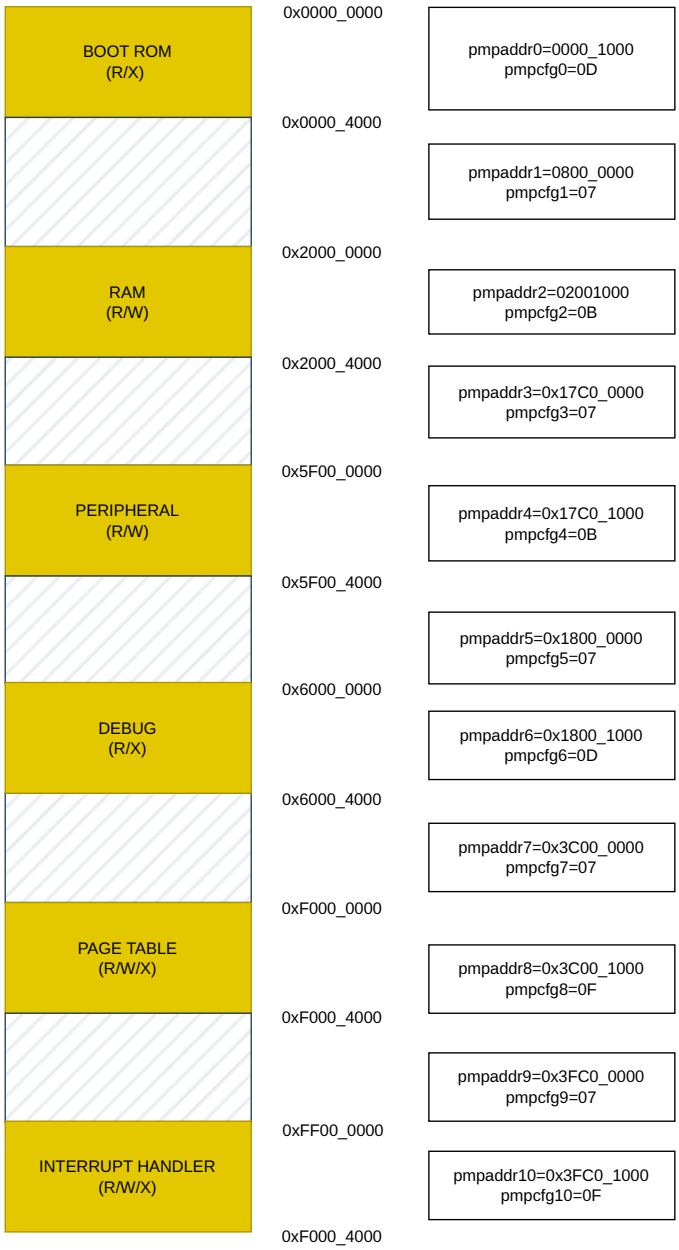
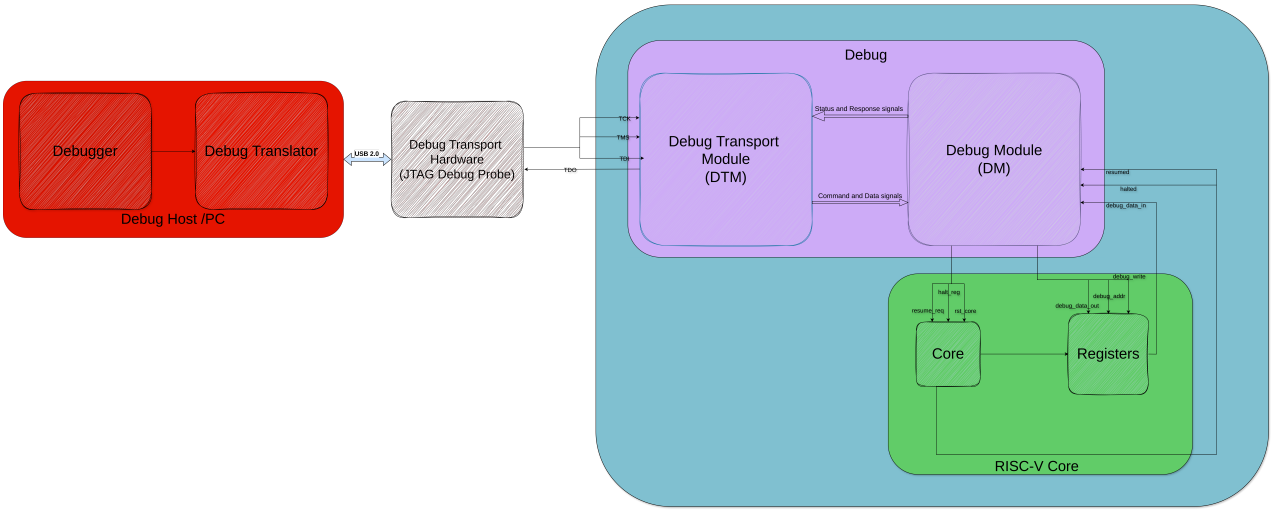


Figure 1.37: Memory map and PMP configuration

1.12 Debugger Design

1.12.1 Debugger Software



Low level JTAG signal Control

The debugger software designed for this work is a custom-built tool developed in Python, specifically tailored to interface with the debug infrastructure of a RISC-V processor through the JTAG interface. Its primary objective is to provide a lightweight yet powerful control interface for hardware-level debugging, offering both interactive control and automation capabilities.

The debugger supports a comprehensive suite of features essential for low-level processor control and visibility. These include core reset functionality, retrieval of core identification and configuration information, and the ability to halt and resume processor execution on demand. Crucially, it enables direct access to architectural state, allowing the user to read from or write to any general-purpose register (GPR), control and status register (CSR), or memory location within the system.

A key component of the debugger is its fine-grained control over the JTAG data transfer process. Unlike high-level tools that abstract away protocol details, this debugger allows for explicit manipulation of JTAG states and data registers, making it highly suitable for testing, validation, and bring-up of the custom debug interface.

To improve usability during halted states, the debugger includes a "quick glance" feature that provides a real-time snapshot of all GPR and CSR values. This functionality enables rapid inspection of the processor state, facilitating efficient debugging sessions and reducing the time to identify incorrect execution paths or corrupted state.

In essence, the debugger functions as both a development and diagnostic tool, bridging the gap between software-driven inspection and hardware-level control in the context of a custom RISC-V debug system.

1.12.2 Debug Translator

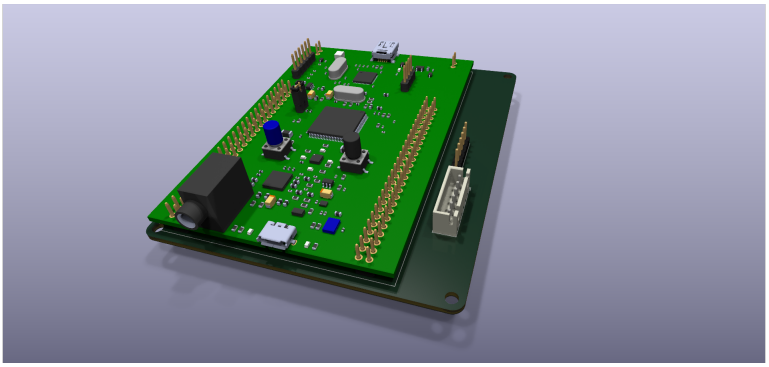


Figure 1.38: Debug Translator

A custom debug translator was developed using the STM32F407 microcontroller to serve as a USB-to-JTAG interface between the host-side debugger and the target RISC-V processor. The translator facilitates reliable communication with transfer rates up to 1 MHz, enabling responsive and efficient debug sessions. The host system interfaces with the STM32-based translator via USB 2.0 Full-Speed using a Virtual COM Port (VCP) interface, which provides a simple and widely supported mechanism for serial communication over USB.

The firmware was written using the STM32 Hardware Abstraction Layer (HAL), ensuring portability across a broad range of STM32 boards. This modular and hardware-agnostic design supports reuse on other compatible STM32 platforms with minimal changes, promoting scalability and adaptability for different deployment scenarios.

Internally, the translator manages the full JTAG state machine, including TMS-driven state transitions and TDI/TDO data shifting. This low-level control enables direct and deterministic communication with the processor's Test Access Port (TAP) and Debug Transport Module (DTM), which is essential for the operation of the Debug Module Interface (DMI).

Overall, the debug translator forms a crucial component in the hardware debug infrastructure, offering a compact, low-cost, and reusable solution for interfacing standard USB hosts with custom RISC-V debug hardware.

1.12.3 JTAG Design

The Joint Test Action Group (JTAG) interface serves as the primary external debug access mechanism for the processor. The implemented JTAG architecture adheres to the IEEE 1149.1 standard, enabling TAP (Test Access Port)-based communication between the host and the debug infrastructure. In this design, the boundary scan registers are intentionally excluded, as the focus is solely on enabling debug and control capabilities rather than full scan-chain testability.

The architecture includes a standard TAP controller, driven by the TCK clock and TMS control signals. The TAP controller is implemented as a finite state machine, with state transitions occurring on the rising edge of TCK, and output signals changing on the falling edge. The controller supports standard transitions for instruction and data scan sequences, with TMS defaulting to logic high (1). Resetting the TAP controller is achieved by applying five consecutive logic high (1) values on TMS, which asynchronously brings the TAP into the Test-Logic-Reset state.

TAP Controller Implementation

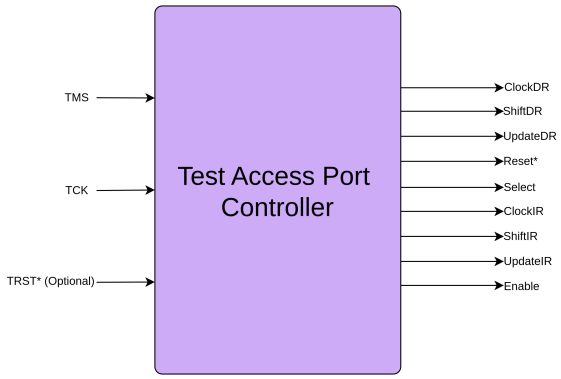


Figure 1.39: TAP Controller

The implemented TAP controller follows the 16-state FSM model defined in the IEEE 1149.1 standard. Key states include Test-Logic-Reset, Run-Test/Idle, Select-DR-Scan, Capture-DR, Shift-DR, Exit1-DR, Pause-DR, Exit2-DR, and Update-DR, along with their instruction register (IR) counterparts. This design ensures deterministic navigation through both instruction and data scan paths using only TCK and TMS.

The state machine transitions synchronously on the positive edge of TCK, while the internal output control signals such as ShiftIR, UpdateIR, ShiftDR, UpdateDR, etc., toggle on the negative edge of TCK to align with standard scan timing protocols. This ensures timing compatibility with standard-compliant external JTAG masters.

The Instruction Register (IR) is composed of:

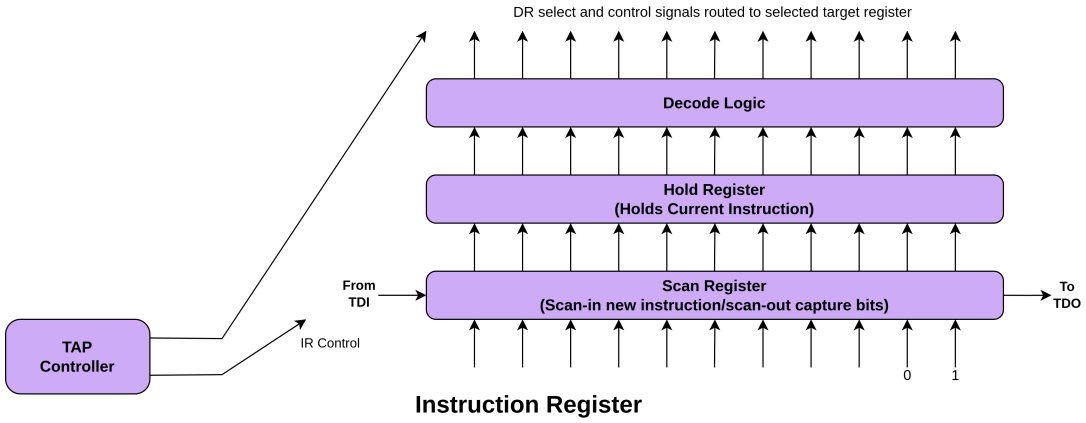


Figure 1.40: Instruction Register

- A scan register, which captures new instructions during the Shift-IR phase
- A hold register, which retains the selected instruction during execution
- Decode logic, which interprets the instruction and selects the appropriate data register (DR) or internal control path

The IR supports the following core instructions:

- BYPASS: Routes the scan path through a single-bit bypass register
- IDCODE: Selects a read-only 32-bit device identification register (if present)
- ACCESS: Enables access to internal debug data registers

During the Capture-IR state, the IR hardwires the two least significant bits to 01 for compliance checking. The IDCODE instruction is loaded by default on reset. If the device does not implement an identification register, the IDCODE behaves as a BYPASS instruction.

The TAP controller emits a set of output control signals:

- ClockDR, ShiftDR, UpdateDR
- ClockIR, ShiftIR, UpdateIR
- Reset, Enable, Select

These signals coordinate the activation of scan paths and synchronize internal operations with external scan sequences. The minimalist implementation avoids unnecessary complexity, ensuring a compact and efficient debug interface.

1.12.4 Overall Debug Flow Architecture

The RISC-V debugger architecture is designed to offer precise control and inspection capabilities over the processor core while maintaining modularity, extensibility, and adherence to the RISC-V Debug Specification (v1.0 Ratified). The implemented design incorporates a minimal but complete debugging system, supporting critical features such as halting and resuming execution, register and memory access, single-step operation, and core status monitoring. Additionally, several custom debug features have been integrated to enable standalone testing of DM without being dependent on a working processor core.

Debugger Initialization

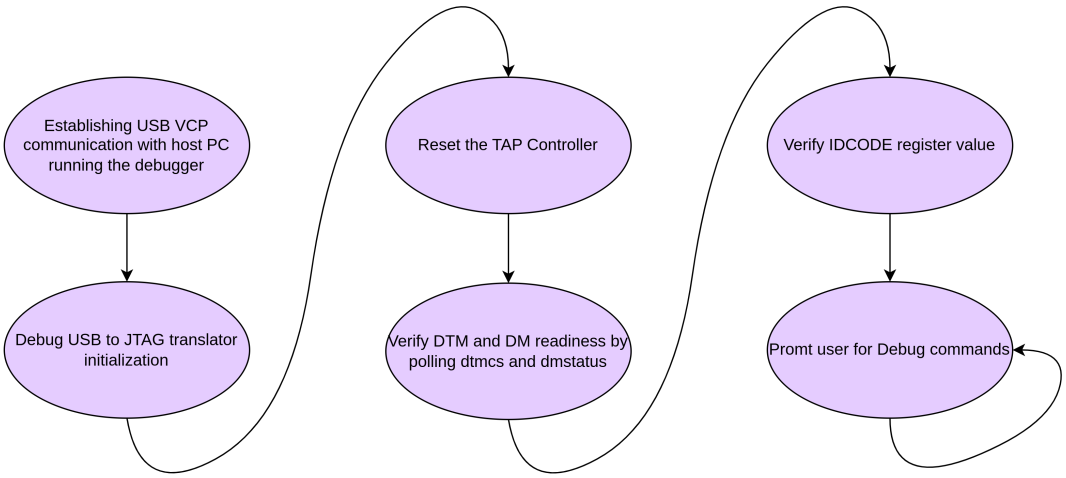


Figure 1.41: Debugger Initialization Steps

At system startup, the debugger initiates a structured initialization sequence to establish communication and prepare the target for debug operations. The process begins by setting up a USB 2.0 Full-Speed virtual COM port (VCP) interface between the host machine and the STM32F407-based USB-to-JTAG translator. This link serves as the communication backbone for transmitting JTAG signals and debug commands.

Once the USB connection is successfully established, the STM32F407 configures its GPIO pins for JTAG signal output (TCK, TMS, TDI, TDO) and performs a hardware reset of the Test Access Port (TAP) controller. The TAP is transitioned from an unknown state to the Test-Logic-Reset state by issuing a sequence of five consecutive logic '1' bits on the TMS line, as per the IEEE 1149.1 JTAG standard. Subsequently, the TAP enters the Run-Test/Idle state, ready for further JTAG operations.

Following TAP initialization, the debugger verifies the readiness of the Debug Transport Module (DTM) and the Debug Module Interface (DMI) by polling the dtmcs (DTM control and status) and dmstatus (Debug Module status) registers, respectively. These checks ensure that the debug infrastructure is responsive and functional. The IDCODE is then read and validated to confirm the identity of the target device.

Once all checks pass, the system is ready to receive debug commands from the user.

Instruction Execution Control

Execution control is a fundamental capability of the debugger. It allows halting the processor, inspecting or modifying state, and resuming execution. These features are implemented through write operations to the dmcontrol register. Halting the core is achieved by asserting the haltreq bit, while resumption is triggered by setting the resumereq bit. Single-stepping is supported by setting the step bit in the dcsr register, which instructs the core to execute exactly one instruction after being resumed. These operations rely on a clean handshake protocol between the Debug Module and the core's debug interface.

State Monitoring

To monitor the processor state, the debugger reads internal registers via abstract commands. These include:

- GPRs and CSRs, accessed using Access Register commands
- Memory locations, accessed using Access Memory commands

The results of such operations are routed through the data0 and data1 registers, which temporarily store the data during DMI transactions. Additionally, core status is obtained from the dmstatus register, which indicates whether the core is halted, running, or reset. The abstractcs register provides status information about the abstract command engine, including error flags (cmderr) and whether the engine is currently busy.

Debug Interaction Flow

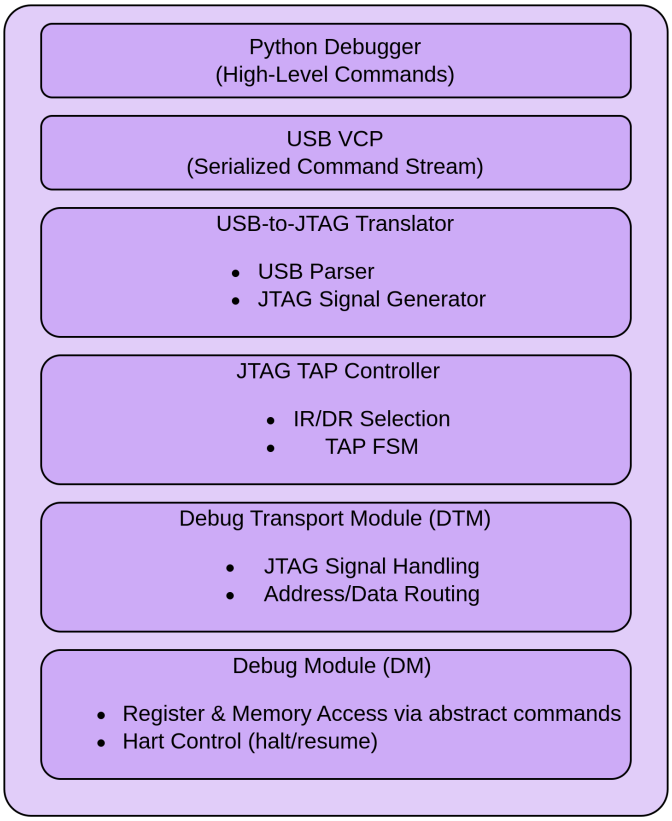


Figure 1.42: Debug hierarchical

The RISC-V debug system implemented in this work follows a hierarchical and modular architecture, enabling clean abstraction between user-level debugging commands and low-level hardware signaling. This layered structure improves scalability, portability, and maintainability, while simplifying system validation and integration.

1. *Debugger Layer (Host Interface):* A custom Python-based debugger forms the top-most layer of the stack, serving as the primary user interface. It allows the user to issue both high-level debug commands (such as halting the core, reading/writing registers, and accessing memory) and low-level JTAG operations. High-level operations are translated into sequences of register accesses to the Debug Module (DM), enabling protocol-agnostic debugging through abstract commands or program buffer execution.

2. *USB Communication Layer (Virtual COM Port):* Communication between the host debugger and the embedded translator is established using a USB 2.0 Full-Speed connection, utilizing the Virtual COM Port (VCP) mode of the STM32F407. This layer is responsible for reliably transmitting encoded command and response packets across the USB link, abstracting the physical transport from the logical operation being performed.

3. *Translator Layer (STM32F407 Microcontroller):* The STM32F407-based translator serves as a USB-to-JTAG bridge. It decodes the received USB commands and

generates the corresponding JTAG waveforms (TCK, TMS, TDI) via its GPIO pins. It also monitors TDO responses for return to the host. The translator manages TAP state transitions, instruction selection, and data register access to interface correctly with the RISC-V TAP and Debug Transport Module (DTM).

4. *JTAG TAP Controller Layer:* The Test Access Port (TAP) controller, implemented according to the IEEE 1149.1 JTAG standard, interprets the JTAG signal stream and handles TAP state machine transitions. It enables access to various scan chains, such as the instruction register (IR) and data register (DR), which are used to issue commands to the Debug Transport Module (DTM) within the target chip.

5. *Debug Transport Module (DTM):* The DTM acts as an interface between the JTAG scan chain and the memory-mapped debug infrastructure. It receives sequences of bits from the JTAG DR and decodes them as register read/write transactions targeting the Debug Module. While sometimes referred to as "DMI transactions", these are not protocol-level messages but rather structured accesses to fixed registers within the DM. The DTM also handles readiness and error signaling to ensure reliable communication.

6. *Debug Module (DM):* The Debug Module is the functional core of the debug system. It implements the actual debug features, such as halting or resuming harts, abstract register/memory access, and execution of instructions via the Program Buffer. It provides memory-mapped registers (dmcontrol, dmstatus, command, dataN, progbufN, etc.) accessible through the DTM. Responses from the DM (e.g., register values or status flags) are returned via the same path, completing the interaction loop.

This layered interaction model offers a robust and flexible framework for RISC-V debugging. By decoupling user-level debug logic from hardware-specific signaling, the design ensures portability across platforms, simplifies development and verification, and enables seamless integration of additional features such as scripting, logging, or GUI-based frontends in the future. Moreover, the modular design facilitates future enhancements—such as support for alternative transport layers or multi-core debug capabilities—without requiring significant changes to the core architecture.

Data Transfer Pathways

The primary mechanism for transferring data between the external debugger and the processor is the DMI interface. Each DMI transaction contains:

- A 8-bit address for the target DM register
- A 32-bit data field
- A 2-bit opcode (e.g., read, write, nop)

These transactions are serialized over JTAG and interpreted by the DTM. Inside the Debug Module, data flows to and from core-facing components via multiplexers and

latches. Abstract commands read from or write to CSRs, memory, or GPRs, depending on the contents of the command register. The results are staged into data0/data1 for read-back or supplied from these registers for write operations.

Error Handling

Robust error handling is built into both the standard and custom components of the debug flow. Abstract command errors are flagged via the cmderr field in abstractcs, which may indicate unsupported access types or illegal operations. The dmstatus register includes bits such as anynonexistent and anyunavailable to report unavailable harts or improper configurations. The debugger software detects protocol-level issues, such as failed TAP sequences or invalid data patterns, and responds by reinitializing the JTAG chain or issuing resets to recover communication.

Implemented Debug Registers

The Debug Module implements a set of essential registers that play critical roles in controlling and querying the debug system's state. These registers manage data transfers, control operations, and provide status feedback for both the debug module and the RISC-V target.

Register	Address	Description
data0	0x04	Primary data register used for reading/writing data between the debugger and the target.
data1	0x05	Used as the address register in abstract memory operations, holding the address for memory accesses.
dmcontrol	0x10	Control interface for halt/resume commands and enabling/disabling the debug module.
dmstatus	0x11	Provides status information about the hart's state (halted, running) and the debug module.
abstractcs	0x16	Status register for the abstract command engine, indicating the availability and status of abstract commands.
command	0x17	Encodes abstract command operations, such as register or memory access requests.
hartinfo	0x12	Contains metadata about the connected hart(s), including available resources and configuration.

Table 1.18: Implemented Debug Registers

Custom Debug Registers

In addition to the standard debug registers, several custom registers were implemented to allow for comprehensive testing of the debugger's functionality. These custom registers provide mechanisms to verify the debugger's operations independently of a valid processor core, supporting fault injection, signal probing, and real-time data monitoring for enhanced debugging validation.

Custom Reg	Address	Function
custom0	0x04	Overrides regno_data used in register read operation
custom1	0x05	Overrides data1_data used in memory read operations
custom2	0x10	Overrides other debug signals from core to debugger
custom3	0x11	Lock register: Enables core bypass when a specific hex key is written
custom4	0x16	Probes real-time values of regno_data
custom5	0x17	Probes real-time values of data1_data
custom6	0x12	Probes other signal lines into the Debug Module

Table 1.19: Custom Debug Registers

These custom registers enable the debugger to be tested and validated independently of the core, ensuring its functionality through various debugging operations. The ability to simulate conditions, inject faults, and monitor real-time data greatly improves the reliability and robustness of the debugger during development.

1.12.5 Debug Module

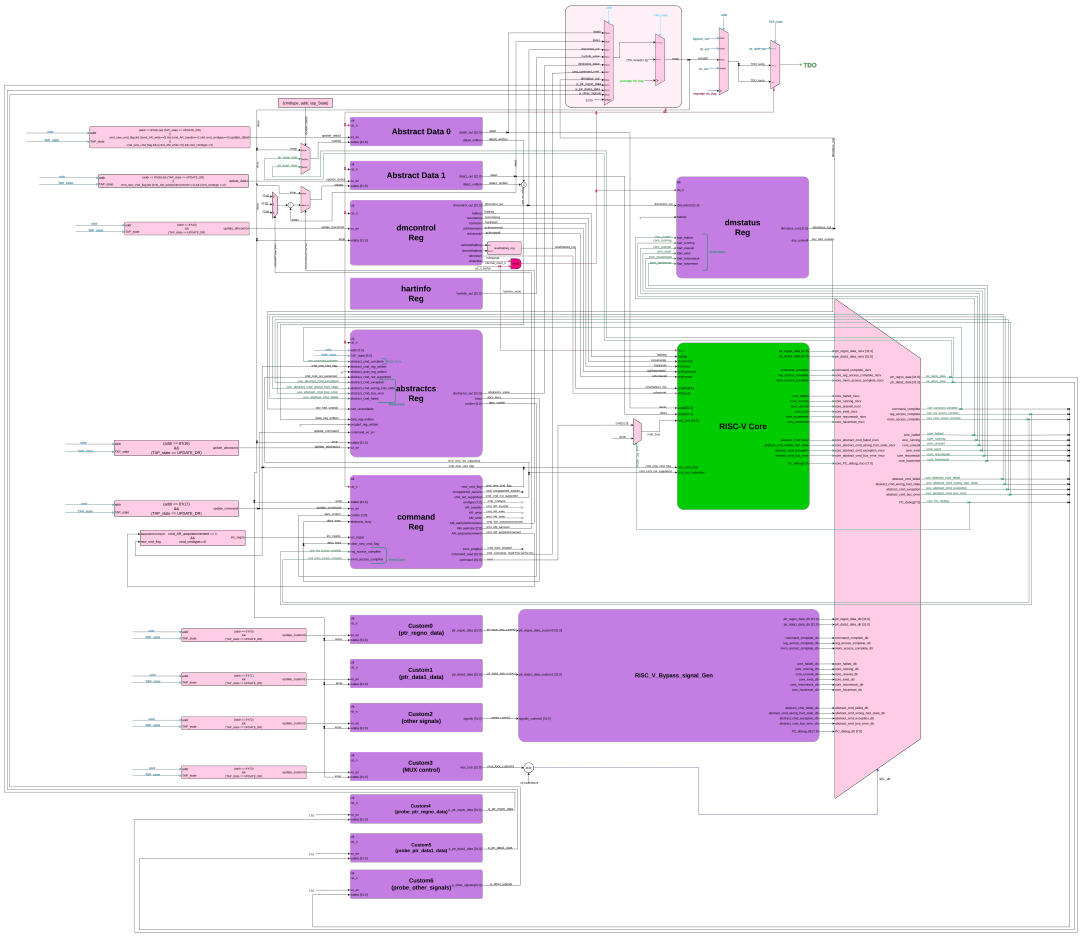


Figure 1.43: Debuger Module Architecture

The Debug Module (DM) is the principal logic block responsible for managing external debugging interactions with the RISC-V processor core. It serves as the terminal endpoint for Debug Module Interface (DMI) transactions and mediates all commands from the host debugger to the internal debug infrastructure. This includes operations such as halting and resuming the core, reading/writing general-purpose and control/status registers (GPRs and CSRs), memory inspection and modification, and runtime monitoring of processor state. The DM design adheres to the RISC-V Debug Specification v1.0 with multiple custom extensions to support self-verification and deep observability for validation purposes.

1.12.5.1 Architecture and Role in the Debug Flow

At a high level, the Debug Module (DM) functions as an intermediary between the Debug Transport Module (DTM) and the RISC-V core and memory subsystem. The DTM receives serialized commands from the external debugger via the JTAG interface

and translates these into simple register read/write operations. The DMI (Debug Module Interface) facilitates communication between the DTM and the DM registers, enabling the transfer of data. Based on the contents of the command register, the DM performs abstract operations such as register or memory access. These operations often involve the data0 and data1 registers, which temporarily hold operand values and results for these operations, allowing for seamless execution of debugging tasks.

1.12.5.2 Implemented Debug Module Registers

The following registers form the core of the debug module:

- **dmcontrol (0x10):** A 32-bit register for managing the DM and harts.
  - dmactive: Master enable for the DM. Must be set to 1 for other fields to be active.
  - ndmreset: Optional field to reset the core or other non-debug elements.
  - haltreq: Request to halt the selected hart.
  - resumereq: Request to resume execution.
  - hartsel: Selects which hart is targeted for debug operations. Only a single-hart system is used in this implementation.
- **dmstatus (0x11):** Reports the status of the DM and selected hart.
  - allhalted, anyhalted: Indicate the halt status of harts.
  - allrunning, anyrunning: Indicate running status.
  - authenticated: Always set to 1 as authentication is not implemented.
  - version: Set to 2, as per RISC-V Debug Spec v1.0.
- **abstractcs (0x16):** Reports the state of the abstract command engine.
  - busy: Indicates whether an abstract command is currently in progress.
  - cmderr: 3-bit error code; cleared on write of 1.
  - datacount: Set to 2, representing availability of data0 and data1.
  - progbufsize: Set to 0, as no program buffer is implemented.
  - support: Reflects that only Access Register (0) and Access Memory (2) are supported.
- **command (0x17):** Writing to this register initiates an abstract command.
  - Supported commands:
    * Access Register: Read/write any CSR or GPR
    * Access Memory: Read/write to main memory (via address and size fields)

- **data0, data1 (0x04, 0x05):** Used as source/destination for operands and results during abstract command execution.
- **hartinfo (0x12):**
  - dataaccess: Set to 1
  - datasize: 2 (32-bit)
  - dataaddr: Address offset to data0. This register helps debuggers understand how to interact with the data registers.

1.12.5.3 Custom Debug Registers

To enable self-validation, system observability, and finer control, several custom debug registers have been implemented:

- **custom0 (ptr_regno_data):** Contains a debug-controlled override value for regno_data, used during testing when bypassing the RISC-V core.
- **custom1 (ptr_data1_data):** Debug-controlled override for data1_data input to the DM.
- **custom2 (control mux signal override):** Encapsulates all multi-bit signals going from the core to the DM, including CSR write enables, GPR access signals, and more. Used to simulate internal events without actual core activity.
- **custom3 (lock register):** Contains a hardcoded unlock code (0xA5A5A5A5). Only when this code is written can the DM bypass the processor's signals and allow custom0, custom1, and custom2 values to take effect. This ensures debug security and prevents unintended bypass activation.
- **custom4, custom5, custom6:** These are monitor-only registers
  - custom4: Observes regno_data from the debugger
  - custom5: Observes data1_data from the debugger
  - custom6: Captures and shows all other control signals being driven into the DM

Together, these registers form a complete observability and override framework allowing full control and debug capability, even without processor cooperation.

1.12.5.4 Abstract Command Pipeline

The debug module's abstract command handler is implemented as a small state machine that:

1. Parses the command (bits [31:24] of command).

2. Validates its legality (e.g., only Access Register or Access Memory).
3. Performs the operation:
    - For register access:
        - Extracts the register number and access type from command bits.
        - Routes the request through internal GPR/CSR muxes or to the custom override.
    - For memory access:
        - Uses the address in data0 and size specifier in command.
        - Performs the read/write over a bus-connected memory interface.

If any invalid command or out-of-bounds register access occurs, the cmderr field is updated accordingly. The command is considered complete when busy is de-asserted.

1.12.5.5 Integration and Control Signals

The DM interfaces with the core via a debug bus that includes:

- GPR index/data buses
- CSR index/data buses
- Control lines: halt, resume, reset
- Status indicators: halted, running

All control signals follow handshake semantics to ensure synchronized behavior across clock domains. This is critical since JTAG-driven inputs are asynchronous relative to the processor clock domain.

The halt/resume logic within the core is implemented using edge-triggered detection of haltreq and resumereq bits in dmcontrol, gated by dmactive. Once halted, the processor enters Debug Mode and begins executing a small debug handler, if configured. Since no program buffer is present, all debug control flows through abstract commands.

1.12.5.6 Debug Flow Summary

The overall debug operation follows this flow:

1. Debugger sends DMI write to dmcontrol → Sets haltreq.
2. Core transitions to halt state → DM updates dmstatus.
3. Debugger issues abstract command (via command + data0).
4. DM executes the command, potentially reading/writing memory/CSRs/GPRs.
5. Debugger clears haltreq and sets resumereq to continue execution.

1.12.5.7 Additional Features and Considerations

- No program buffer is implemented. All debug activity is performed via data registers and abstract commands.
- Single-step functionality is supported using the dcsr.step field and custom halt/resume logic.
- No authentication or access protection mechanisms are currently implemented.
- The system supports a single hart only (hartsel=0).

1.12.5.8 Additional Features and Considerations

The implemented Debug Module supports all required functionalities defined by the RISC-V Debug Specification and extends into several optional capabilities to enhance flexibility and control. Specifically, the module:

1. Exposes implementation details such as supported abstract commands, data register sizes, and hart configuration through hartinfo, abstractcs, and related control/status registers.
2. Supports halting and resuming of the processor through the haltreq and resumereq signals in dmcontrol.
3. Reports core status via fields in dmstatus, indicating whether the hart is halted or running.
4. Enables abstract register access to all general-purpose registers (GPRs) of the core via the command, data0, and data1 registers.
5. Provides access to a core reset signal, enabling debug initialization immediately after system reset.
6. Implements early debug entry, allowing the debugger to connect and halt the hart as it exits reset, regardless of the reset cause.
7. Extends abstract access to non-GPR registers, including CSRs, via properly encoded command fields and internal decode logic.
8. Supports memory-mapped debug access, allowing read and write operations to the processor's memory address space from the debugger's perspective.

These features ensure that the debugger can not only inspect and manipulate core state, but also validate behavior from power-up through full execution, including support for advanced use cases like bare-metal debugging and post-reset diagnostics.

1.12.6 Reset Control and Debug Entry Coordination

Reset control is a critical feature in the debug architecture, enabling the debugger to reliably manage the processor's execution lifecycle and gain visibility from the very first instruction after reset. The implemented debug system supports two complementary reset mechanisms—one initiated via the Debug Module and the other through an external hardware switch—each integrated to ensure flexibility, safety, and effective debug entry.

The Debug Module-based reset is triggered by the external debugger through the dmcontrol register. This software-controlled reset signal is internally synchronized and routed to the processor core and associated debug logic. Upon release of the reset, the Debug Module can assert a haltreq signal, allowing the processor to enter debug mode immediately after reset. This supports the RISC-V Debug Specification's requirement for "debug-after-reset" behavior. The mechanism ensures that the debugger can observe and intervene before the processor executes the first instruction post-reset, enabling use cases such as boot-time diagnostics, breakpoint setup, or register probing.

Parallel to this, the system includes a manual reset mechanism via a physical push-button switch that triggers a global hardware reset. This reset line asynchronously resets all major subsystems, including the core, memory interface, and debug infrastructure. While this method ensures a clean and full system reinitialization, it requires the debugger to re-establish the debug connection and reconfigure debug state (e.g., reasserting halt requests or restoring context). To distinguish between these reset causes, the debug logic inspects reset signals and can maintain internal state only when reset is initiated via the debug interface.

In both scenarios, reset coordination is handled carefully to avoid metastability or inconsistent states. All resets are synchronized with system clocks, and transitions between reset and active states are controlled via explicit state machines. The implementation ensures that TAP controller integrity is maintained across reset cycles, and all required Debug Module registers are reinitialized or retained appropriately depending on the reset source.

This dual reset mechanism, combining controlled debugger-triggered reset with full-system hardware reset, provides comprehensive control for development, testing, and fault recovery, making the debug system robust and responsive to diverse use cases.

1.12.7 Run Control

Run control is the core mechanism through which the external debugger can orchestrate and manage the execution state of the processor. It encompasses halting and resuming the core, initiating single-step execution, and coordinating transitions between normal and debug modes. In compliance with the RISC-V Debug Specification, and tailored to a single-hart implementation, the debug module provides fine-grained control over the processor's execution state through a combination of hardware signals and abstract commands.

The primary mechanism for halting the processor is the assertion of the haltreq bit within

the dmcontrol register. When this bit is set by the external debugger, the Debug Module asserts an internal halt request signal to the core. The core responds by entering debug mode at a well-defined point in execution and sets the allhalted bit in the dmstatus register, which serves as confirmation to the debugger that the halt was successful. The halted state allows the debugger to read or write general-purpose registers (GPRs), control and status registers (CSRs), and initiate memory transactions through the abstract command interface.

Resuming execution is initiated by setting the resumereq bit in the dmcontrol register. Upon detecting this request, the Debug Module clears the haltreq and generates a resume signal to the core. The core then exits debug mode and resumes normal execution from the address stored in the dpc (Debug Program Counter) register. The allrunning bit in dmstatus is used to acknowledge that the core has resumed successfully.

In addition to full halt/resume control, the system supports hardware single-step execution. When the step bit in the dcsr (Debug Control and Status Register) is set, and the processor is resumed from debug mode, it executes exactly one instruction and automatically re-enters debug mode. This behavior allows for precise instruction-level debugging without external breakpoints or complex instrumentation. The debugger can read the updated dpc after each step to trace program flow in a fine-grained manner.

The core's entry into debug mode can also be triggered via exceptions or external interrupts, depending on the configuration in the dcsr. However, in this implementation, debug entry is typically coordinated via explicit requests from the debugger to ensure deterministic behavior and full external control.

All control signals between the Debug Module and the core are synchronized and safely latched to avoid glitches or metastable conditions. These include haltreq, resumereq, ack_halted, ack_running, and single_step. Custom logic ensures that only one control path (e.g., debugger vs. external reset) dominates at any time, preventing conflicts in state transitions.

This robust run-control infrastructure ensures reliable interaction between the debugger and the processor, enabling essential debug capabilities such as program inspection, register probing, and step-by-step execution control.

1.12.8 Abstract Command Execution and Data Register Handling

The RISC-V Debug Module implements abstract commands to allow an external debugger to perform basic operations such as reading/writing general-purpose or control/status registers, and accessing memory. These commands are encoded in the command register and executed when the core is in debug halt mode.

The command register is a 32-bit register that encodes the command type and associated control fields. Two types of abstract commands have been implemented in this design: Access Register (cmdtype = 0x00) and Access Memory (cmdtype = 0x02). Execution of a command may involve one or both of the Data Registers, data0 and data1, to pass

arguments and results.

1.12.8.1 Access Register (cmdtype = 0x00)

This command enables access to the core's general-purpose or CSR registers. It uses the following format:

Bits	Field Name	Description
0-15	regno	Register number to be accessed
16	write	If set, writes data0 to the register; if clear, reads from register into data0
17	transfer	If set, initiates the actual register transfer
18	postexec	If set, the program buffer (not implemented here) would execute after transfer
19	aarpostincrement	If set, regno is incremented after each access
20-22	aarsize	Indicates register size (e.g., 2 for 32-bit)
23	reserved	Must be zero
24-31	cmdtype	Should be 0x00 for Access Register

Table 1.20: Bit fields of the command register for Access Register (cmdtype = 0x00)

Operational Flow:

- When transfer is set, the DM interprets the regno field, and performs the register access.
- If write is set, data0 is written to the target register.
- If write is clear, the value from the register is loaded into data0.
- aarpostincrement, when set, increments regno after the transfer, enabling batch operations.
- The aarsize field should match the register width of the target (typically 2 for 32-bit).

1.12.8.2 Access Memory (cmdtype = 0x02)

This command provides abstract memory access from the hart's perspective. It is useful for loading and storing values in memory-mapped locations, assuming the core is halted.

Bits	Field Name	Description
0-14	reserved	Must be zero
14-15	target-specific (reserved)	Reserved for future use
16	write	If set, writes value from data0 to memory; else reads into data0
17-18	reserved	Must be zero
19	aampostincrement	If set, memory address (in data1) is incremented after each access
20-22	aamsize	Indicates access size (0 = 8-bit, 1 = 16-bit, 2 = 32-bit, etc.)
23	aamvirtual	Set to 0; only physical addresses supported
24-31	cmdtype	Should be 0x02 for Access Memory

Table 1.21: Bit fields of the command register for Access Memory (cmdtype = 0x02).

Operational Flow:

- The base memory address is supplied in data1.
- If write is set, the content in data0 is written to the memory location.
- If write is clear, the value at the memory location is loaded into data0.
- aampostincrement, if set, increments the memory address in data1 post-access.
- The aamsize field determines the access granularity (byte/half-word/word).
- Only physical addresses are supported (aamvirtual = 0).

1.12.8.3 Data Registers

Two data registers are implemented:

- data0: The primary register for passing and receiving data during abstract command execution.
- data1: Used primarily for memory commands, representing the memory address (or secondary data in multi-operand instructions).

Data registers serve as operand channels between the debugger and the debug module. Their contents are valid only while the core is halted. Care is taken to ensure all abstract command execution completes before resuming the core to avoid data corruption.

1.12.9 Debug Module Registers

The Debug Module (DM) implements a set of memory-mapped registers through which an external debugger can manage and monitor core activity. These registers conform to the RISC-V Debug Specification and are accessed via the Debug Module Interface (DMI). The implemented registers allow core control, status reporting, abstract command execution, and customized debugging workflows. All registers are addressable via the DMI address space, with specific control and status bits determining the flow of debug interactions.

1.12.9.1 dmcontrol (Address:0x10):

The dmcontrol register is the principal control mechanism for managing core-level debug operations via the Debug Module Interface (DMI). This 32-bit register enables halt/resume requests, hart selection, and system/hart-level resets. All interactions between the debugger and the core are mediated through this register, making it critical to the functionality of the external debug interface.

Bits	Field Name	Description
0	dmactive	Enables the Debug Module. Must be set to 1 before any debug operation. Clearing this bit resets the DM to its initial state.
1	ndmreset	Initiates a non-debug module reset of the target system. This is typically used to reset peripherals or the system while keeping the debug module active.
2	clrresethaltreq	Clears any previously set halt request that was configured to trigger upon reset.
3	setresethaltreq	When set, ensures that the selected hart will enter halt state immediately upon reset. Useful for debugging from the first instruction after a system restart.
4	clrkeepalive	Clears the keep-alive request, which prevents the debugger from timing out due to inactivity.
5	setkeepalive	Enables the keep-alive mechanism to maintain communication with the debugger even during long operations.
6-15	hartselhi	Upper bits of the hart selection field. Used in combination with hartsello to identify specific harts in multi-core systems.
16-25	hartsello	Lower bits of the hart selection field. Together with hartselhi, forms the full hart selection signal.
26	hasel	Hart array select enable. When set, enables hart selection through hartselhi and hartsello.
27	ackunavail	When set, acknowledges that the selected hart is currently unavailable (e.g., powered down or inaccessible).
28	ackhavereset	Acknowledges that the hart has recently undergone a reset. This bit is typically set by the hardware and cleared by the debugger.
29	hartreset	Initiates a direct reset of the selected hart. This is distinct from ndmreset, which targets the system.
30	resumereq	Requests that the selected hart resume execution from the halted state.
31	haltreq	Requests that the selected hart enter the halted state. This is the primary means by which the debugger takes control of a running core.

Table 1.22: Bit fields and description of dmcontrol register

Design Considerations and Implementation Notes:

Activation: All debug control activity is gated by the dmactive bit. This must be set before any requests such as halt, resume, or reset take effect.

- **Reset Handling:** Two reset pathways are supported: one via ndmreset (for full system reset without affecting the debug module), and another via hartreset (for

selective hart-level reset). Both are implemented in coordination with an external physical reset switch for complete board-level reinitialization.

- **Halt on Reset:** The combination of setresethaltreq and clrresethaltreq enables or disables the ability to force a core to halt immediately after a reset—critical for debugging from the first instruction.
- **Single-Hart Operation:** Although this implementation targets a single-hart processor, the hart selection fields (hartselhi, hartsello, hasel) are retained for compliance with the RISC-V Debug Specification and future scalability.
- **No Keepalive Fields:** The setkeepalive and clrkeepalive fields described in the specification are not implemented, as no timeout or heartbeat mechanism is currently required in the system.

1.12.9.2 dmstatus (Address: 0x11):

The dmstatus register is a 32-bit read-only status register that reflects the real-time operational state of the Debug Module and the associated hart(s). It provides essential insight into the availability, running/halted state, and reset status of the core, as well as metadata about debug infrastructure implementation. This register is queried regularly by the external debugger to synchronize with the core's debug state.

Bits	Field Name	Description
0-3	version	Indicates the version of the debug specification supported. A value of 3 indicates compliance with Debug Spec v1.0
4	confstrptrvalid	Always 0 in this implementation. Indicates whether a valid configuration string pointer is available. Not implemented.
5	hasresethaltreq	Set if the Debug Module supports halting the hart immediately after a reset. Implemented and functional.
6	authbusy	Indicates that authentication is in progress. Not implemented in this design (hardwired to 0).
7	authenticated	Indicates whether the debugger is authenticated. Not implemented (hardwired to 1).
8	anyhalted	Set if any hart is currently in a halted state. Always reflects the state of the single hart in this design.
9	allhalted	Set if all selected harts are halted. For single-hart systems, same as anyhalted.
10	anyrunning	Set if any hart is running.
11	allrunning	Set if all selected harts are running.
12	anyunavail	Set if any selected hart is unavailable (e.g., powered down). Not expected to be set in this system.
13	allunavail	Set if all selected harts are unavailable.
14	anynonexistent	Set if any selected hart index does not correspond to an implemented hart. Useful in multi-hart configurations; here, unused.
5	allnonexistent	Set if all selected harts are nonexistent.
16	anyresumeack	Indicates that any selected hart has acknowledged a resume request.
17	allresumeack	Indicates that all selected harts have acknowledged a resume request.
18	anyhavereset	Indicates that any hart has been reset since dmcontrol.dmactive was last asserted.
19	allhavereset	Indicates that all selected harts have experienced a reset.
20-21	Reserved	Hardwired to 0.
22	impebreak	Set if the core can halt due to an ebreak instruction in debug mode. Not enabled in this design.
23	stickyunavail	Sticky indication that some selected hart was unavailable at some point since last clear. Cleared only by dmactive deassertion.
24	ndmresetpending	Indicates that a system reset (ndmreset) is pending. Set by dmcontrol.ndmreset.
25-31	Reserved	Hardwired to 0.

Table 1.23: Bit fields and description of dmstatus register

Design Notes:

- **Single-Hart Simplification:** In this single-core design, the any* and all* fields naturally mirror each other.
- **Reset Tracking:** The anyhavereset and allhavereset fields help verify whether a reset has occurred after dmactive was last set, which is crucial for debugging post-reset behavior.
- **No Authentication:** Since this implementation operates in a trusted and controlled environment, no authentication mechanism is required. Accordingly, authbusy is hardwired to 0, and authenticated is hardwired to 1.
- **impebreak Support:** This implementation does not support halting the core via the ebreak instruction in debug mode. Consequently, the impebreak field is hardwired to 0. Debug entry is achieved through external control via the debug module (e.g., haltreq) rather than software breakpoints.

The dmstatus register serves as a vital tool for observing and coordinating the debug flow and ensures that the external debugger can operate in a synchronized and state-aware manner.

1.12.9.3 hartinfo (Address: 0x12):

The hartinfo register provides static information about a given hart's debug capabilities and configuration. In a multi-hart system, this register would offer per-hart debug support characteristics; however, in this single-hart implementation, it conveys basic information applicable to the lone core.

Bits	Field Name	Description
0-11	dataaddr	Base address offset of the first data register (typically data0). This field is valid and reflects the actual location used by the debug logic.
12-15	datasize	Indicates the number of implemented data registers. In this implementation, only data0 and data1 are present, hence datasize = 1 (meaning 2 registers).
16	dataaccess	Indicates whether the data registers are memory-mapped and accessible outside of abstract commands. Not supported in this design; the field is hardwired to 0.
17-19	Reserved	Reserved for future use. Must be zero.
20-23	nscratch	Number of implemented dscratch registers. This design does not implement any dscratch registers; the field is zero.
24-31	Reserved	Reserved for future use. Must be zero.

Table 1.24: Bit fields and description of hartinfo register

Design Notes:

- **Data Registers:** Only two data registers (data0, data1) are implemented, and they are exclusively used within abstract commands for operand and result exchange.
- **No dscratch:** The optional scratch registers (dscratch0, dscratch1) are not implemented, as all transient data handling during debug is managed internally or via custom registers.
- **Non-Memory-Mapped Data Access:** The data registers are not memory-mapped for external visibility and are instead accessed exclusively through debug commands (e.g., access register, access memory).

This lean configuration simplifies the debug module design while remaining compliant with essential portions of the RISC-V Debug Specification.

1.12.9.4 abstractcs (Address: 0x16):

The abstractcs (Abstract Control and Status) register provides information about the capabilities of the abstract command interface, as well as real-time status of the abstract command execution engine. It plays a central role in managing the execution of abstract commands such as register and memory access operations.

Bits	Field Name	Description
0-3	datacount	Indicates the number of implemented data registers available for abstract commands. In this implementation, two registers are implemented (data0, data1), hence datacount = 2.
4-7	Reserved	Reserved for future use. Must be zero.
8-10	cmderr	Indicates the result of the last abstract command. The values are encoded as: 0 = success 1 = busy 2 = cmd not supported 3 = exception 4 = halt/resume error 5 = bus error 6 = reserved 7 = failed due to other reasons The debugger is expected to clear this field by writing 1s to it.
11	relaxedpriv	Indicates whether abstract commands are permitted to access registers across privilege boundaries. Not supported in this design (hardwired to 0).
12	busy	Set to 1 if an abstract command is currently executing. This design sets and clears this bit as needed during command handling.
13-23	Reserved	Reserved for future use. Must be zero.
24-28	progbufsize	Indicates the size of the implemented program buffer. As the program buffer is not implemented in this design, this field is hardwired to 0.
29-31	Reserved	Reserved for future use. Must be zero.

Table 1.25: Bit fields and description of abstractcs register

Design Notes:

- **Program Buffer:** This design does not implement a program buffer, and all abstract operations are executed directly via the debug module logic.
- **Error Handling:** The cmderr field plays a critical role in communicating failure states such as unsupported commands or unresponsive core behavior. Errors are latched until explicitly cleared by the debugger.
- **Busy Flag:** During execution of abstract commands, the busy flag is set to prevent concurrent command issuance. It is cleared once the command completes or fails.
- **Simplicity and Conformance:** By omitting complex features like relaxedpriv and program buffer, the implementation remains lightweight while fulfilling required functionality as per the RISC-V Debug Specification.

1.12.9.5 command (Address: 0x17):

The command register is used by the debugger to issue abstract commands to the debug module. It defines both the type of operation to perform and the parameters necessary to execute that operation. In this implementation, only Access Register and Access Memory command types are supported (corresponding to cmdtype = 0x0 and cmdtype = 0x2 respectively). The command register is 32 bits wide and must be written before setting the dmcontrol.haltreq bit or issuing a command through the abstract interface.

Bits	Field Name	Description
0-15	regno	Register number to be accessed
16	write	If set, writes data0 to the register; if clear, reads from register into data0
17	transfer	If set, initiates the actual register transfer
18	postexec	If set, the program buffer (not implemented here) would execute after transfer
19	aarpostincrement	If set, regno is incremented after each access
20-22	aarsize	Size of the register access: 2 = 32-bit (as used in this design).
23	reserved	Must be zero
24-31	cmdtype	Should be 0x00 for Access Register

Table 1.26: Bit fields and description of the command register for Access Register (cmdtype = 0x00)

Bits	Field Name	Description
0-14	reserved	Must be zero
14-15	target-specific	Reserved for future use
16	write	1 = Write operation, 0 = Read.
17-18	reserved	Must be zero
19	aampostincrement	Increments memory address after access if set.
20-22	aamsize	Indicates access size (0 = 8-bit, 1 = 16-bit, 2 = 32-bit, etc.)
23	aamvirtual	Set to 0, as all accesses are to physical memory.
24-31	cmdtype	Should be 0x02 for Access Memory

Table 1.27: Bit fields and description of the command register for Access Memory (cmdtype = 0x02).

Execution Semantics:

- **Command Lifecycle:** A command is written to the command register, and its associated operands (such as register or memory address) are placed in data0 and/or data1. The command execution begins immediately and sets the abstractcs.busy bit.

- **Completion and Error Reporting:** Upon completion, the busy bit is cleared and the result is indicated in abstractcs.cmderr. If a command fails due to an invalid regno, unsupported size, or bad cmdtype, appropriate error codes are latched.
- **Post-Increment:** This feature allows automated iteration through consecutive registers or memory addresses, simplifying multi-word transactions.

1.12.9.6 data0, data1 (Addresses: 0x04–0x05):

The data0 and data1 registers are 32-bit wide memory-mapped registers used as the primary data interface between the debugger and the target core through the debug module. These registers are part of the abstract command mechanism and are used for both register and memory access operations.

Purpose and Usage

- **data0:** This is the primary data register used for transferring values to/from the RISC-V core or memory.
- **data1:** This auxiliary register is primarily used to hold memory addresses during abstract memory access commands.

These registers are critical for read and write transactions initiated through the command register. Their content is interpreted based on the cmdtype field in the command, which determines whether the access is targeted at a core register or memory.

Register	Width	Purpose	Access Context
data0	32-bit	Main operand/result data register	Register and Memory Access
data1	32-bit	Memory address or auxiliary data	Mainly Memory Access

Table 1.28: Description of dataN registers

Implementation Notes

- Both registers are implemented with simple latching mechanisms.
- They are not cleared after command execution and retain their last value until overwritten.
- No locking mechanism is enforced; coherence is ensured by command flow control and the busy signal in abstractcs.

1.12.9.7 Custom Registers (custom0 to custom6) (Addresses: 0x70–0x76):

To support staged bring-up and validation of the debugger independently of the RISC-V core, a set of custom debug registers (custom0 through custom6) has been integrated. These registers implement two essential mechanisms:

1. **Bypass:** Simulates responses from the RISC-V core for abstract commands, enabling the debugger to function even in the absence of a running core.
2. **Probing:** Observes and captures signal values as they arrive at the Debug Module from the core, without interfering with normal operation.

Both mechanisms operate simultaneously, ensuring that while the debugger can function using simulated inputs, the actual signal paths from the core can still be verified for correctness and consistency.

Register	Role	Description
custom0	Core Bypass (Access Register)	Bypasses the data read from a register (regno) when executing Access Register abstract commands.
custom1	Core Bypass (Access Memory)	Bypasses the memory data read from the address pointed by data1 during Access Memory commands.
custom2	Core Bypass (Other Signals)	Bypasses all non-data signals from the core to the DM, such as halted, cmd_err, busy, etc.
custom3	Lock / Core Bypass Selector	Acts as a security lock: core bypass mode is only enabled if this register holds a specific pre-defined hex sequence (e.g., 0xABCD1234). Prevents accidental core bypass activation.
custom4	Probe (Access Register Data)	Probes the data being routed to the DM from the core as a result of an Access Register command.
custom5	Probe (Access Memory Data)	Probes the data read from memory (via data1) that is routed to the DM from the core for Access Memory commands.
custom6	Probe (Other Signals)	Probes additional debug-related signals flowing from the core to the DM—mirrors what custom2 bypasses but from the DM's observation point.

Table 1.29: Description of the custom registers

Simultaneous Bypass and Probing:

The design allows bypass mode (enabled via custom3) and signal probing to operate in parallel:

- When the debugger is in bypass mode, it relies entirely on values from custom0, custom1, and custom2 to respond to abstract commands.

- At the same time, custom4, custom5, and custom6 do not interfere with signal flow but passively record what the core would have sent to the DM, allowing for diagnostic and verification use.

This design ensures:

- Debugger development and testing can proceed independently of core integration.
- Real-time visibility into core-DM interactions, improving traceability and observability.
- Bypass logic does not suppress or overwrite actual core signals; both views coexist for comprehensive testing.

1.13 Debug Core integration

1.13.1 Debug Mode Entry

When the debugger requests for the debug entry, the processor enters in the debug mode. Currently we have three main debug entry points.

1. ***Haltreq* goes high (Async debug entry):** If the processor is in normal operation (pipeline operation) and it detects the *haltreq* signal, then we also enter the Debug state if the processor is not freezed. As soon as we enter the debug mode, *pending_ async_ dbg* goes low.
2. *resethaltreq* goes high (Reset debug entry)
3. Single Stepping
4. EBREAK (Sync debug entry)

Debug Mode is a special processor mode used only when a hart is halted for external debugging. Because the hart is halted, there is no forward progress in the normal instruction stream. How Debug Mode is implemented is not specified here.

Following things happen during debug mode entry:

1. Kills all the instructions in the pipeline.
2. All interrupts are masked using *interrupt_ dbg_ global* signal and also by writing mstatus[MIE] = 1'b0
3. All traps are disabled
4. All the debug instructions are executed as they do in M-mode except control instructions (**Branch and Jumps**) irrespective of their destination address, all PC related instructions like **AUIPC** act as illegal istructions.

1.13.2 RESET

If the *halt* signal (driven by the hart's halt request bit in the Debug Module) or *hasresethaltreq* are asserted when a hart comes out of reset, the hart must enter Debug Mode before executing any instructions, but after performing any initialization that would usually happen before the first instruction is executed.

1.13.3 HALT

Before entering the debug mode, the processor is halted when it encounters any of the debug entry indication through CSR or through external signals. In HALT mode, we do following things:

1. *cause* is updates with the cause of debug
2. *prv* is set to 3 and *v* is set to 0 to indicate Machine Mode.
3. *dpc* is set to the next valid instruction which needs to be executed when the core leaves the debug mode.
4. Kill all the instructions in the pipeline
5. write mstatus[MIE] <= 1'b0, to disable all the Software, timer and external interrupts.
6. Assert *interrupt_ dbg_ global* signal
7. PC <- dm_halt_addr (Program buffer)

When the processor is halted, it waits for signal from debugger that whether it has to execute the abstract command or start the execution from the program buffer. If processor wants to single step the instruction, then the debugger has to write the DCSR[step] = 1'b1 using the abstract command and then it can go in the single stepping mode. If processor recevies the *resumereq_ i*, then it goes back to the functional (normal runnning) mode.

If while executing the program buffer EBREAK occurs, then the PC goes to the starting of program buffer and the processor is again halted. All the interrupts are disbaled in the HALT mode.

1.13.4 DEBUG EXIT SEQUENCE

When we get *resumereq_ i* as 1'b1, following sequence is followed.

1. Fetch PC is retored with the one from *dpc*.
2. Previllage levels are restores as it was before entering the debug mode.
3. Leave the debug mode.

1.13.5 Single Steping

This method is only available to external debuggers, and is the preferred way to single step.

1. If the external debugger sets the ***step*** bit before resuming a halted hart, the processor will execute exactly one instruction before automatically re-entering Debug Mode.
2. If a trap (e.g., exception or interrupt) occurs while executing the instruction, the processor will enter Debug Mode right after the PC is updated to the trap handler address. The *mtval* and *mcause* CSRs will be updated however, the trap handler is not executed; it's immediately halted again.
3. If the executed instruction changes the PC to a location that would cause an exception (e.g., instruction fetch fault), that exception is deferred and will only be taken next time the hart resumes.

1.13.6 Debug FSM

Figure 1.44 shows the debug FSM implemented in the core.

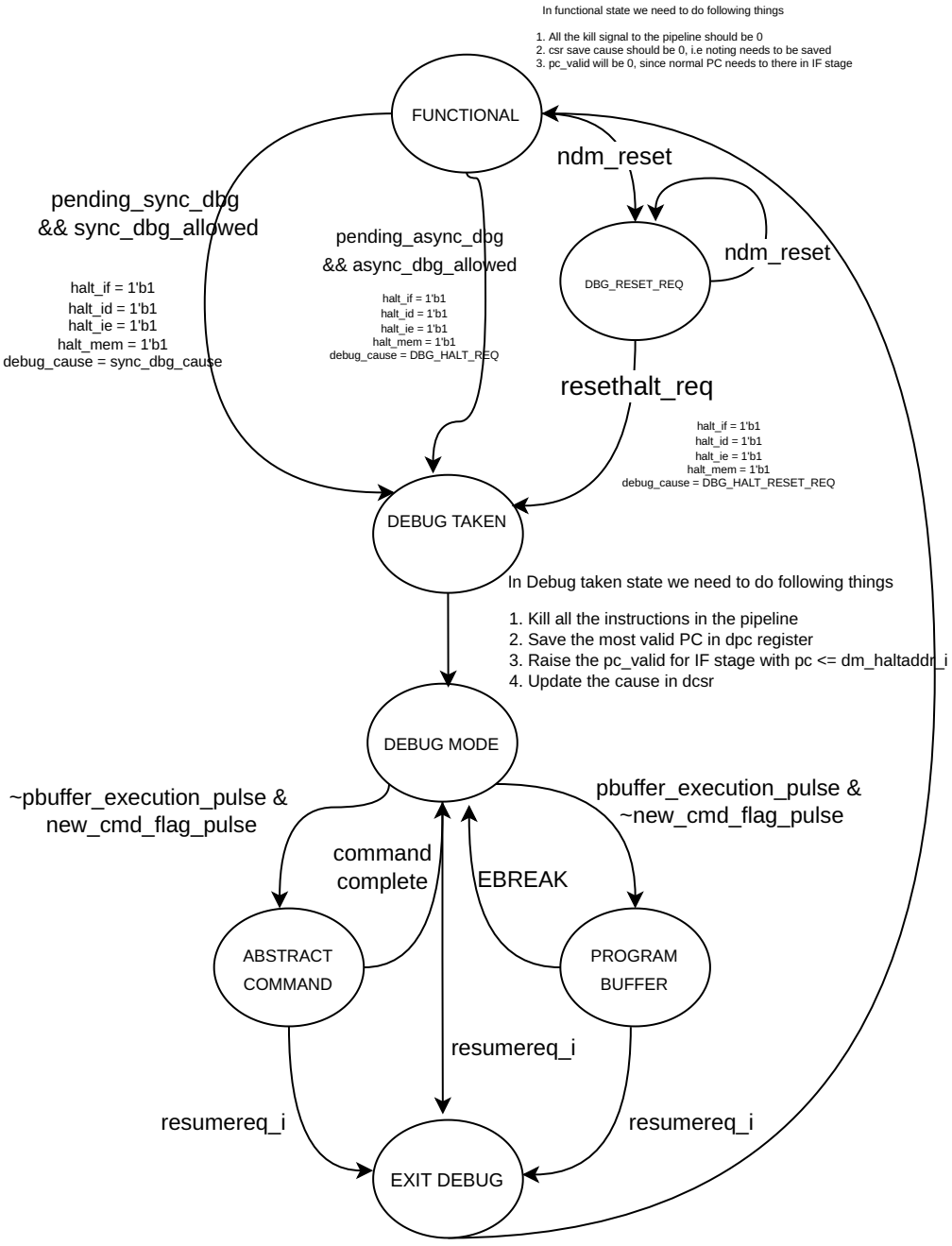


Figure 1.44: Debug FSM

FUNCTIONAL State: Upon coming out of reset, the processor checks the *haltreq_i* signal. If this signal is not asserted, the processor enters the FUNCTIONAL state. In this state, the processor behaves normally—fetching instructions from instruction memory, executing them through the pipeline, and writing results back to the register file or memory. Entry into debug mode is not permitted while the processor is stalled, such as during memory latency (ICACHE/DCACHE) or FPU operations. Even if debug requests such as *ndmreset* or *haltreq_i* are asserted during this time, the FSM defers action until the processor becomes idle and resumes normal flow. This behavior ensures that the processor only transitions into debug when it is safe to do so, preserving pipeline integrity and consistency.

DBG_RESET_REQ State:

If a non-debug module reset (*ndmreset*) is received while in the FUNCTIONAL state, the FSM transitions to the DBG_RESET_REQ state. In this state, the processor remains in reset, in accordance with the debug spec, until the *ndmreset* signal is deasserted. Once the reset is lifted, the FSM checks the *resethaltreq_i* signal. If this is asserted, indicating a request to enter debug mode immediately after reset, the FSM transitions to the DBG_TAKEN state. Otherwise, it stays in DBG_RESET_REQ or returns to FUNCTIONAL depending on other pending conditions. This mechanism allows a debugger to take control of the processor directly out of reset. ***DBG_TAKEN State:***

The DBG_TAKEN state handles the initiation of a debug session. The FSM transitions here if *haltreq_i* is asserted or if an appropriate debug entry condition is detected (e.g., from a triggered breakpoint, step event, or ebreak instruction). In this state, the cause of the debug event is written into the dcsr.cause CSR as defined in the RISC-V Debug Specification. If the debug cause is not due to single-step execution, the currently executing program counter is stored in the *dpc* register. The processor's PC is then loaded with the *dm_halt_address*, which points to the beginning of the debug program buffer, and the *debug_mode* signal is asserted. From here, the FSM transitions into the DBG_MODE state, indicating that the processor is now fully in debug mode and halted for external inspection or instruction.

DBG_MODE State:

In the DBG_MODE state, the processor is halted and remains idle, waiting for instructions from the debug host. It allows the debugger to inspect and modify state or initiate specific debug actions. If the debug module receives a *new_cmd_flag*, the FSM transitions to the ABSTRACT_COMMAND state to execute abstract commands, which may include register access or memory operations. Alternatively, if the *pbuffer_execution* signal is asserted, the FSM enters the PROGRAM_BUFFER state to begin executing a sequence of instructions from the program buffer. The FSM stays in DBG_MODE as long as no execution requests are pending, providing a safe and passive state for debug interaction.

ABSTRACT_COMMAND State:

The ABSTRACT_COMMAND state manages the execution of abstract commands issued via the debug module interface (DMI). These commands are typically used to read/write registers or access memory. The processor executes these commands one at a time

through the instruction pipeline. Once a command is successfully executed, the FSM sets the command_complete flag and returns to DBG_MODE to await the next command. If a resume request (*resumereq_i*) is detected during or after command execution, the FSM prepares to exit debug mode and transitions to the DBG_EXIT state.

PROGRAM_BUFFER State:

In the PROGRAM_BUFFER state, the processor begins executing instructions from a predefined buffer area (*dm_halt_address*) loaded during the DBG_TAKEN state. This mode is used for running short debug programs, such as test sequences or small memory routines. Execution continues until an EBREAK instruction is encountered, which serves as an exit condition. Upon reaching EBREAK, the FSM transitions back to DBG_MODE to await further commands. Similar to the ABSTRACT_COMMAND state, if *resumereq_i* is asserted during program buffer execution, the FSM will instead transition to DBG_EXIT.

DBG_EXIT State:

The DBG_EXIT state handles the safe exit from debug mode. Here, the PC is restored with the value saved in the *dpc* register, effectively resuming execution from the instruction where the core had been halted. The *debug_mode* signal is deasserted, and the FSM returns to the FUNCTIONAL state, resuming normal operation. This transition ensures that any modifications made during debug mode are correctly applied, and program control is seamlessly handed back to the running application.